

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Кафедра ЕОМ



КУРСОВИЙ ПРОЕКТ

з дисципліни «Системне програмне забезпечення»
на тему: «Розробка програмного забезпечення для захисту від вірусів та
шкідливих програм»

Варіант № 33

Студента 3-го курсу групи:
KI-304
123 «Комп'ютерна інженерія»
Стафесв А. А.
Керівник
доц. каф. ЕОМ
Олексів М. В.

Львів - 2026

ЗАВДАННЯ НА КУРСОВИЙ ПРОЕКТ

Тема проекту: Розробка програмного забезпечення для захисту від вірусів та шкідливих програм (антивірусний сканер «Antivirus MVP»).

Розробити програму - антивірусний сканер, що забезпечує захист персонального комп'ютера від вірусів та шкідливих програм шляхом сигнатурного аналізу файлів. Утиліта повинна здійснювати пошук загроз у вибраній користувачем директорії на основі порівняння MD5-хешів файлів з локальною базою сигнатур, а також надавати користувачеві повний контроль над усіма діями програми.

Утиліта повинна мати такий функціонал:

- Сигнатурне сканування обраної директорії з обчисленням MD5-хешу кожного файлу та порівнянням з локальною базою сигнатур `signatures.db`;
- Відображення списку результатів сканування з індикацією прогресу та підсумковою статистикою (перевірено файлів / виявлено загроз);
- Перегляд списку активних процесів операційної системи (PID, ім'я процесу, шлях до виконуваного файлу) через Windows API;
- Ізоляція виявлених загроз до карантину з можливістю відновлення або остаточного видалення;
- Збереження текстового звіту про результати сканування;
- Підтримка білого списку (виключень) зі сканування;
- Журналювання всіх операцій програми у текстовий файл;
- Простий та зрозумілий графічний інтерфейс користувача.

Утиліта повинна бути орієнтованою на операційну систему сімейства Windows (Windows 10 версії 2004 та новіші, Windows 11).

Мова написання програми – утиліти: C++17 із застосуванням фреймворку Qt 6 (Qt Widgets). Низькорівневу складову бізнес-логіки оформлено у вигляді статичної бібліотеки, рівень представлення – у вигляді GUI-застосунку.

АНОТАЦІЯ

У курсовому проекті розроблено програмне забезпечення для захисту персональних комп'ютерів від вірусів та шкідливих програм - антивірусний сканер «Antivirus MVP» для операційної системи Windows. Робота виконана за результатами аналізу п'яти існуючих рішень (Windows Security, ESET, ClamWin, rkhunter, System Informer), що дозволив виявити дефіцит легких, прозорих та контрольованих користувачем інструментів захисту.

Програму реалізовано мовою C++ 17 із застосуванням фреймворку Qt 6 Widgets за модульною архітектурою: низькорівневе ядро бізнес-логіки оформлене у вигляді статичної бібліотеки, рівень представлення - у вигляді GUI-застосунку. Реалізовано сигнатурне сканування файлів на основі MD5-хешування (QCryptographicHash, QDirIterator), перегляд активних процесів системи через Windows API (CreateToolhelp32Snapshot), ізоляцію виявлених загроз до карантину, генерацію текстових звітів, потоко-безпечне журналювання та підтримку білого списку виключень. Тривалі операції виконуються в окремому потоці (QThread), що забезпечує реагуючий інтерфейс.

Розробку організовано за методологією Scrum (14 User Stories, 3 спринти, 19 Story Points) із використанням системи контролю версій Git за стратегією Gitflow. Працездатність підтверджено набором із 12 тест-кейсів, що покривають функціональні та нефункціональні вимоги; коректність виявлення загроз перевірено за допомогою стандартного тестового файлу EICAR.

ЗМІСТ

Завдання на курсовий проект.....	2
Анотація.....	3
Зміст	4
Вступ	6
1. Аналіз існуючих рішень та методів захисту від шкідливого ПЗ.....	8
1.1. Огляд методів виявлення шкідливого програмного забезпечення	8
1.2. Аналіз існуючих програмних аналогів	8
1.2.1. Вбудований захист Windows Security	8
1.2.2. Програмний комплекс ESET	9
1.2.3. Антивірусний сканер ClamWin.....	10
1.2.4. Сканер rkhunter.....	11
1.2.5. Утиліта моніторингу System Informer	12
1.3. Обґрунтування мети та завдань курсового проєкту	12
2. Проектування	14
2.1. Технічне завдання	14
2.1.1. Найменування та область застосування	14
2.1.2. Мета та завдання розробки	14
2.1.3. Характеристика предметної області та обґрунтування необхідності розробки	14
2.1.4. Функціональні вимоги до системи	15
2.1.5. Нефункціональні вимоги до системи.....	16
2.1.6. Допущення та обмеження	18
2.1.7. Ризики та план мітигації.....	19
2.4.1. Макет інтерфейсу користувача (Wireframe).....	26
3. Реалізація	28
3.1. Планування розробки (User Stories / Scrum)	28
3.2. Структура проєкту та конфігурація збірки.....	29
3.3. Реалізація ядра системи.....	30
3.3.1. Модуль Logger.....	30
3.3.2. Модуль SignatureDB	31
3.3.3. Модуль Scanner	32
3.3.4. Модуль Quarantine	32
3.3.5. Модуль ProcessList.....	33
3.3.6. Модуль ReportWriter.....	35
3.4. Реалізація графічного інтерфейсу	35
3.5. База сигнатур та демонстраційний файл EICAR	36

4. Тестування на інструкція користувача	37
4.1. Стратегія тестування	37
4.1.1. Мета та підхід.....	37
4.1.2. Тестове середовище	37
4.1.3. Тестові дані.....	38
4.1.4. Критерії успішності тестування	38
4.2. Результати виконання тест-кейсів.....	38
4.3. Виявлені дефекти	42
4.4. Інструкція користувача.....	42
4.4.1. Встановлення та запуск	42
4.4.2. Виконання сканування	43
4.4.3. Управління карантинном.....	44
4.4.4. Перегляд процесів.....	45
4.4.5. Налаштування білого списку	46
4.4.6. Оновлення бази сигнатур	47
Висновки.....	48
Список використаної літератури	49
Додатки	50
Додаток А. Лістинг програми	51
Додаток Б. Блок-схема алгоритму	82
Додаток В. Діаграма варіантів використання утиліти.....	83
Додаток Г. Діаграма класів програмного забезпечення.....	84

ВСТУП

Стрімкий розвиток інформаційних технологій та масштабне поширення цифрових систем у всіх сферах суспільного життя зумовлюють невідпинне зростання кіберзагроз. Шкідливе програмне забезпечення залишається одним із найбільш поширених інструментів зловмисників, завдаючи значної шкоди як приватним користувачам, так і корпоративному середовищу. У відповідь на це комерційні розробники антивірусних рішень впроваджують дедалі складніші евристичні алгоритми та моделі машинного навчання, спрямовані на виявлення як відомих, так і раніше невідомих загроз.

Проте така агресивна автоматизація породжує власну проблему: сучасні антивірусні системи нерідко функціонують як непрозорий «чорний ящик», приховуючи від користувача логіку своїх рішень. Для технічних спеціалістів - розробників, тестувальників, системних адміністраторів - це створює практичні труднощі: хибні спрацьовування на щойно скомпільований або нестандартний код, автоматичне видалення файлів без пояснення причин та відсутність гнучкого контролю над правилами виявлення. Комерційні рішення оптимізовані для масового користувача і не передбачають прозорого режиму роботи, придатного для середовища розробки та тестування.

З огляду на це, актуальним є створення легкого модульного програмного забезпечення, яке поєднує базові механізми захисту з повною прозорістю прийнятих рішень та гнучким контролем з боку користувача. Такий інструмент не претендує на заміну комерційних антивірусних систем, проте закриває конкретну нішу: забезпечення діагностичного та керованого захисту в середовищі технічних спеціалістів.

Метою курсового проєкту є розробка програмного забезпечення «Antivirus MVP» для захисту систем на базі операційної системи Windows, що поєднує механізми сигнатурного аналізу файлів із моніторингом активних процесів у реальному часі та надає користувачеві повний контроль над управлінням виявленими загрозами.

Для досягнення зазначеної мети в межах проєкту вирішуються такі

завдання: дослідження існуючих підходів до виявлення шкідливого програмного забезпечення; проєктування модульної архітектури застосунку; реалізація підсистеми сигнатурного аналізу на основі хеш-порівняння; реалізація підсистеми моніторингу активних процесів засобами Windows API; розробка інтерфейсу користувача з детальним журналюванням подій та можливістю налаштування виключень.

Курсовий проєкт складається зі вступу, чотирьох розділів, висновків, списку використаної літератури та додатків. У першому розділі проведено аналіз існуючих антивірусних систем і методів захисту. У другому розділі сформовано технічне завдання, нефункціональні вимоги та спроектовано архітектуру програмного комплексу. Третій розділ присвячено безпосередній реалізації ядра мовою C++ та розробці графічного інтерфейсу на базі фреймворку Qt 6. У четвертому розділі наведено результати тестування системи за визначеними тест-кейсами та інструкцію користувача.

1. АНАЛІЗ ІНСУЮЧИХ РІШЕНЬ ТА МЕТОДІВ ЗАХИСТУ ВІД ШКІДЛИВОГО ПЗ

1.1. Огляд методів виявлення шкідливого програмного забезпечення

Проблема захисту від шкідливого ПЗ вирішується багат шаровим гібридним підходом, оскільки жоден окремий метод не забезпечує достатню ефективність. Основні методи:

Сигнатурний аналіз - порівняння файлів із базою відомих вірусних сигнатур. Переваги: швидкість, точність. Недоліки: неефективний проти невідомих загроз.

Евристичний аналіз - розпізнавання підозрілого поведінкового коду на основі статичного аналізу. Виявляє поліморфні віруси, що видозмінюють себе.

Поведінковий моніторинг - відстеження дій процесів у реальному часі (читання файлів, системні виклики, мережева активність). Дозволяє блокувати загрози нульового дня.

Хмарна репутація - централізований аналіз невідомих файлів та їх оцінка на основі глобальної телеметрії мільйонів комп'ютерів.

1.2. Аналіз існуючих програмних аналогів

Для обґрунтування необхідності розробки було проаналізовано п'ять різних рішень для захисту від шкідливого ПЗ, що представляють різні підходи та цільові аудиторії.

1.2.1. Вбудований захист Windows Security

Опис: Вільний, інтегрований у Windows 10/11 антивірус корпорації Microsoft. Використовує сигнатури, евристику, машинне навчання та компонент Smart App Control для оцінки репутації програм.

Переваги: Безплатний, безшовна інтеграція з ОС, мінімум конфліктів, постійні оновлення.

Недоліки: Smart App Control блокує безпечні портативні програми без дорогих цифрових сертифікатів; слабкість механізму Tamper Protection; монополізація системних ресурсів Antimalware Service Executable.

1.2.2. Программный комплекс ESET

Переваги: Висока швидкість реакції на нові загрози, низьке споживання ресурсів, комплексний інструментарій (шифрування, батьківський контроль).

Недоліки: Критична проблема - SSL-фільтрація за замовчуванням розриває з'єднання npm, pip та Git через самопідписаний сертифікат; надмірна підозрілість до системних утиліт.

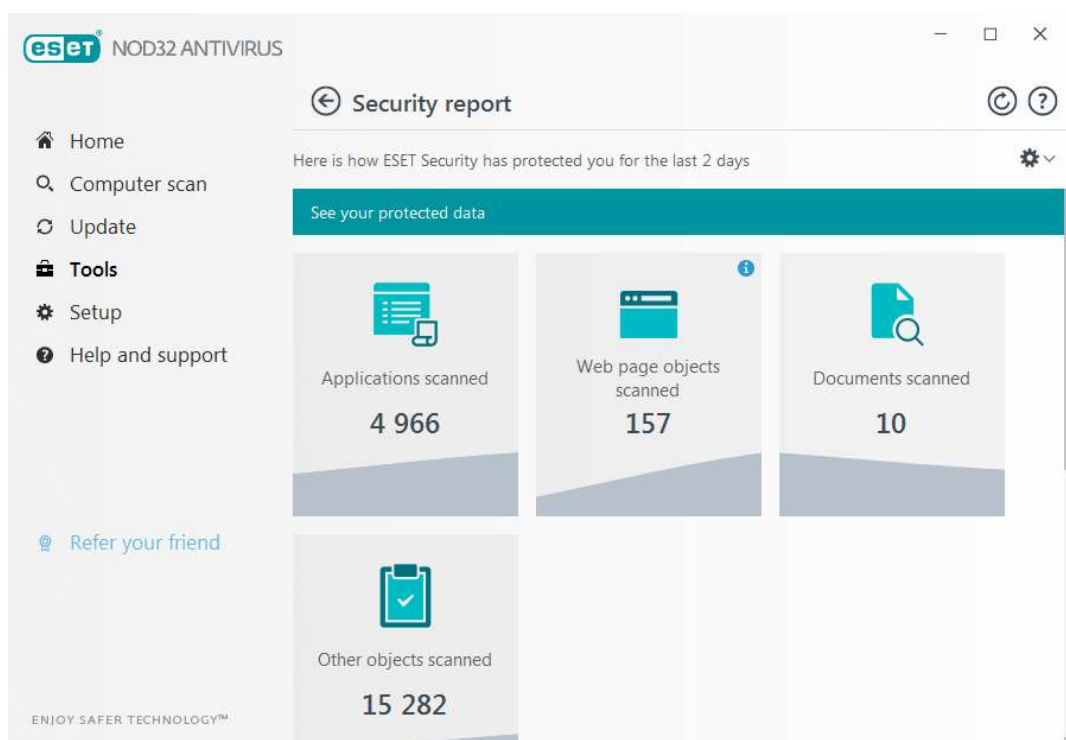


Рис. 1.2. Програма ESET

1.2.3. Антивірусний сканер ClamWin

Опис: Безплатний open-source сканер на основі рушія ClamAV. Класичний сигнатурний аналіз без резидентного моніторингу.

Переваги: Повна прозорість вихідного коду (GPL v2), нульовий вплив на продуктивність, працює на старих системах (Windows 98/XP).

Недоліки: Відсутність моніторингу у реальному часі; архаїчний інтерфейс; повільне повне сканування; слаба евристика проти поліморфних загроз.

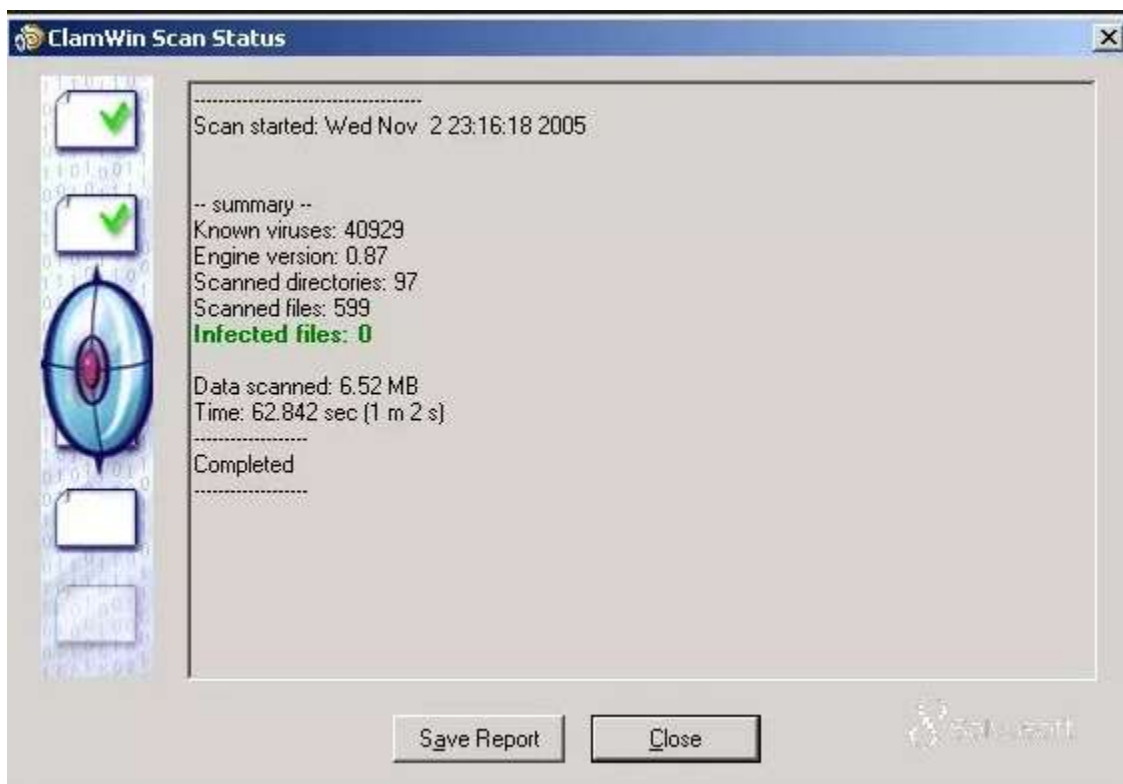


Рис. 1.3. Програма ClamWin

1.2.4. Сканер rkhunter

Опис: Спеціалізована утиліта для виявлення руткітів та бекдорів. CLI-інтерфейс, криптографічна верифікація системних файлів. Розроблена для UNIX/Linux.

Переваги: Унікальна здатність виявляти ядерні загрози; глибокий аудит цілісності системи; відкритий код, портативність.

Недоліки: Відсутність графічного інтерфейсу; велика кількість хибних спрацьовувань при оновленнях ОС; вузька спеціалізація не замінює класичний антивірус.

```

Checking for rootkits...

Performing check of known rootkit files and directories
XOR.DDoS - Rootkit           [ Warning ]
55808 Trojan - Variant A     [ Not found ]
ADM Worm                     [ Not found ]
AjaKit Rootkit               [ Not found ]
Adore Rootkit                [ Not found ]
aPa Kit                      [ Not found ]
Apache Worm                  [ Not found ]
Ambient (ark) Rootkit        [ Not found ]
Balaur Rootkit               [ Not found ]
BeastKit Rootkit             [ Not found ]
beX2 Rootkit                 [ Not found ]
BOBKit Rootkit               [ Not found ]

```

Рис. 1.4. Програма rkhunter

1.2.5. Утиліта моніторингу System Informer

Опис: Безплатна open-source утиліта (раніше Process Hacker) для низькорівневого моніторингу процесів та ручного аналізу динамічної активності системи.

Переваги: Феноменальна деталізація поведінки ОС; незамінна для дослідження інцидентів; повна прозорість та портативність.

Недоліки: Повна відсутність автоматизації - користувач сам вирішує, який процес небезпечний; надмірна складність інтерфейсу; часто блокується антивірусами як підозрілий.

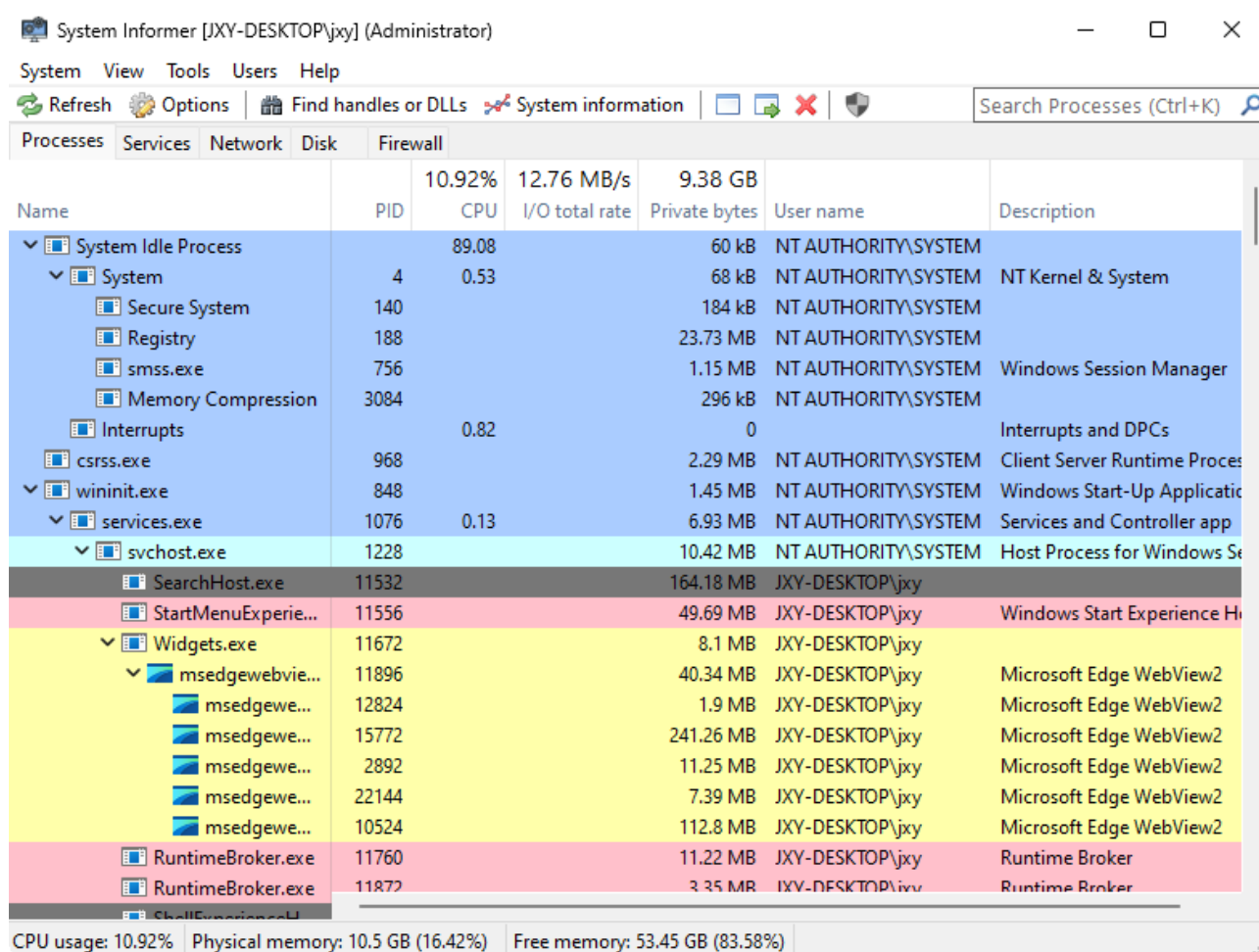


Рис. 1.5. Програма System Informer

1.3. Обґрунтування мети та завдань курсового проєкту

Аналіз п'яти рішень виявив системну проблему сучасної антивірусної індустрії: кожен інструмент проєктується як моноліт з заздалегідь визначеною філософією, жертвуючи гнучкістю заради спеціалізації.

Кумулятивні недоліки аналогів:

- Автоматичні системи (Windows Security, ESET) перевищують контроль користувача й блокують легітимний код;
- Open-source рішення (ClamWin) не мають моніторингу у реальному часі;
- Спеціалізовані інструменти (rkhunter, System Informer) недоступні для рядових користувачів (CLI, відсутність автоматизації);
- Всі рішення постійно генерують хибні спрацьовування або ускладнюють легітимну роботу розробників;
- Немає простого, легкого, прозорого рішення, що поєднує переваги.

Запропоноване рішення: Розробити легкий, модульний антивірусний сканер з ясным UI, що поєднує:

- Сигнатурний аналіз з динамічним завантаженням баз;
- Евристичний модуль для виявлення поліморфних загроз;
- Моніторинг активних процесів у реальному часі;
- Мінімальне споживання ресурсів;
- Повна прозорість механізмів рішень (log-файли, деталізація);
- Ручний контроль користувача над автоматизацією.

2. ПРОЕКТУВАННЯ

2.1. Технічне завдання

2.1.1. Найменування та область застосування

Назва проекту: Модульна система виявлення та моніторингу шкідливого програмного забезпечення на базі Windows.

Область застосування: Розробка легкого, контрольованого антивірусного сканера для персональних комп'ютерів з операційною системою Windows 10/11, призначеного як для рядових користувачів, так і для системних адміністраторів та розробників ПЗ.

2.1.2. Мета та завдання розробки

Мета: Створити прозорий, легкий антивірусний сканер з графічним інтерфейсом на базі Qt/C++, що реалізує сигнатурне сканування файлів, перегляд активних процесів та ізоляцію загроз у карантин, з повним контролем користувача над всіма діями програми.

Основні завдання: 1. Розробити модуль сигнатурного аналізу: обхід директорії, обчислення MD5-хешу кожного файлу та порівняння з локальною базою signatures.db; 2. Реалізувати вікно перегляду активних процесів системи (PID, ім'я, шлях до бінарника); 3. Розробити модуль карантину: переміщення виявлених загроз до ізольованої директорії з можливістю відновлення або видалення; 4. Розробити графічний інтерфейс (Qt Widgets) з вкладками: Сканування, Процеси, Карантин, Журнал; 5. Реалізувати логування всіх операцій програми у файл antivirus.log; 6. Підготувати демонстраційну базу сигнатур з EICAR-тестовим файлом.

2.1.3. Характеристика предметної області та обґрунтування необхідності розробки

За результатами аналізу п'яти провідних рішень виявлено критичний дефіцит на ринку: - Комерційні антивірусні комплекси (ESET, Windows Security) обмежують контроль користувача або порушують працездатність легітимних засобів розробки; - Open-source рішення (ClamWin) не мають моніторингу у реальному часі та виглядають архаїчно; - Спеціалізовані інструменти (rkhunter, System Informer) недоступні для рядових користувачів або потребують глибоких

технічних знань.

Запропоноване рішення має заповнити цю нішу збалансованим, прозорим, контрольованим інструментом з можливостями професійного рівня та простотою використання для звичайних користувачів.

2.1.4. Функціональні вимоги до системи

2.1.4.1 Функційність модуля сканування

BP1-Сканування: Програма повинна виконувати сканування обраної директорії з порівнянням MD5-хешів файлів з локальною базою сигнатур. - Детально: Через кнопку “Обрати директорію” (стандартний QFileDialog) користувач вибирає папку; запускається рекурсивний обхід (QDirIterator); для кожного файлу обчислюється MD5-хеш через QCryptographicHash; хеш порівнюється з signatures.db; результати (чисто/загроза) відображаються у таблиці QTableWidgetItem; прогрес показує QProgressBar; після завершення - рядок зі статистикою (перевірено/знайдено загроз).

BP2-Карантин: Програма повинна переміщувати виявлені загрози до ізольованої директорії карантину. - Детально: При виявленні загрози та підтвердженні користувача програма переміщує файл через QFile::rename() до директорії %APPDATA%\AntivirusMVP\Quarantine\; у вкладці “Карантин” відображається список ізольованих файлів (ім'я, оригінальний шлях, дата ізоляції); для кожного запису доступні кнопки “Відновити” та “Видалити назавжди”.

BP3-Перегляд процесів: Програма повинна відображати список всіх активних процесів системи. - Детально: Вкладка “Процеси” показує таблицю з колонками: PID, Ім'я процесу, Шлях до виконуваного файлу; список отримується через Windows API CreateToolhelp32Snapshot() / Process32First() / Process32Next(); кнопка “Оновити” оновлює список; для підозрілих процесів (не мають цифрового підпису або шлях у Temp) підсвічується рядок червоним кольором.

BP4-Звітування: Програма повинна записувати результати сканування у текстовий файл. - Детально: Після завершення сканування програма автоматично зберігає звіт у

%APPDATA%\AntivirusMVP\Reports\scan_YYYYMMDD_NHMMSS.txt; звіт містить: дату/час, кількість проаналізованих файлів, список виявлених загроз з повними шляхами та MD5-хешами; кнопка “Відкрити звіт” відкриває файл у стандартному текстовому редакторі.

BP5-Білий список: Програма повинна підтримувати список виключень зі сканування. - Детально: У вкладці “Налаштування” є текстове поле для додавання шляхів-виключень (по одному на рядок); список зберігається у %APPDATA%\AntivirusMVP\whitelist.txt; перед скануванням кожного файлу перевіряється, чи його шлях не входить до виключень - якщо так, файл пропускається.

BP6-Журналювання: Програма повинна вести журнал усіх операцій у текстовий файл. - Детально: Усі події (запуск сканування, знаходження загрози, переміщення до карантину, помилки читання файлів) записуються до %APPDATA%\AntivirusMVP\antivirus.log з часовою міткою та рівнем (INFO / WARNING / ERROR); вкладка “Журнал” завантажує та відображає вміст файлу через QPlainTextEdit; реалізується через singleton-клас Logger.

2.1.4.2 Функційність модуля інтерфейсу

BP7-Графічний інтерфейс: Програма повинна мати зрозумілий графічний інтерфейс на Qt Widgets з чотирма вкладками. - Детально: Головне вікно QMainWindow містить QTabWidget з вкладками: “Сканування” (вибір директорії, прогрес, таблиця результатів), “Процеси” (таблиця активних процесів), “Карантин” (список ізолюваних файлів), “Журнал” (текстове поле з логом) та “Налаштування” (білий список); всі тривалі операції виконуються у QThread, щоб UI залишався реагуючим.

2.1.5. Нефункціональні вимоги до системи

2.1.5.1 Вимоги до продуктивності

НВ1-Швидкість сканування: Час сканування одного типового файлу (розмір 1 МБ) не повинен перевищувати 50 мс. - Обґрунтування: Дозволяє завершити повне сканування типової домашньої папки (100 ГБ, ~100 тис. файлів) протягом ~1.5 години при фоновому режимі.

НВ2-Утилізація CPU: Під час фонового сканування програма не повинна

утилізувати більше 25% CPU однієї ядра. - Обґрунтування: Не перешкоджає основній роботі користувача при відкритих браузерях, редакторах та інших застосунків.

НВ3-Утилізація пам'яті: Резидентна утилізація оперативної пам'яті не повинна перевищувати 150 МБ у звичайному режимі. - Обґрунтування: Залишає достатньо ресурсів для комфортної роботи користувача на системах з 8 ГБ оперативної пам'яті.

НВ4-Час відповіді інтерфейсу: Час відповіді на дії користувача (клацання кнопок, прокручування) не повинен перевищувати 100 мс. - Обґрунтування: Забезпечує плавний та реагуючий користувацький досвід.

2.1.5.2 Вимоги до надійності та безпеки

НВ5-Надійність: Програма не повинна вихід з ладу при обробці пошкоджених або аномальних файлів. - Обґрунтування: Повинна мати обробку виключень для всіх потенційно небезпечних операцій (читання файлів, парсинг структур); критична помилка не повинна призвести до краху програми.

НВ6-Захист від втручання: Програма повинна захищатися від спроб вірусів видалити або модифікувати її файли. - Обґрунтування: Виконуваний файл програми та файли конфігурації повинні встановлюватися з обмеженнями доступу; мають перевірятися цифрові підписи всіх критичних компонентів.

НВ7-Конфіденційність: Програма не повинна збирати та передавати дані про користувача без його явної згоди. - Обґрунтування: Всі операції сканування, логування та оновлення баз виконуються локально; хмарні функції (якщо реалізовані) активуються тільки після явного підтвердження користувачем.

2.1.5.3 Вимоги до операційного середовища

НВ8-Операційна система: Програма повинна працювати на Windows 10 версії 2004 та новішої, а також Windows 11 всіх версій. - Обґрунтування: Охоплює 95% встановлених копій Windows на світі (станом на 2026 рік).

НВ9-Мінімальні технічні вимоги: - Процесор: Intel Core i3 або еквівалент AMD, 2 ядра; - ОЗУ: мінімум 4 ГБ; - Вільне місце на диску: мінімум 500 МБ; - Мережевий доступ: для завантаження оновлень баз.

НВ10-Мови програмування та технології: - Мова реалізації: C++ 17; -

Фреймворк GUI та утиліти: Qt 6 (Qt Widgets, QFileDialog, QThread, QCryptographicHash, QDirIterator, QJsonDocument); - **Windows API:** tlhelp32.h - для отримання списку процесів (CreateToolhelp32Snapshot); - **Формат бази сигнатур:** плаский текстовий файл signatures.db (MD5-хеш + ім'я загрози, одна на рядок); - **Інструменти білду:** CMake 3.20+ з Qt6 CMake-інтеграцією або Qt Creator (qmake); - **Система керування версіями:** Git з Gitflow стратегією; - **Тестування:** Qt Test (вбудований фреймворк Qt для unit-тестів).

2.1.6. Допущення та обмеження

2.1.6.1 Допущення

- Користувач має права адміністратора на своєму комп'ютері для встановлення програми та запуску драйверів;
- Комп'ютер має стабільне мережне з'єднання для завантаження оновлень баз (мінімум 1 Мбіт/с);
- База вірусних сигнатур розпорядника буде оновлюватися щомісяця з новим контентом;
- Не враховуються атаки користувача з правами TrustedInstaller або спеціальні техніки ескалації привілеїв;
- Програма розроблюється як рішення для захисту кінцевих точок (endpoint protection), а не для централізованого корпоративного управління.

2.1.6.2 Обмеження

- Функціональність за межами проекту (виключена зі скоупу MVP):
 - Автоматичне планування сканувань за розкладом - відсутній планувальник задач;
 - Блокування запуску підозрілих процесів у реальному часі - відсутній драйвер рівня ядра (minifilter);
 - Мережеве оновлення бази сигнатур - база оновлюється вручну заміною файлу;
 - Інтеграція з контекстним меню Windows Explorer;
 - Евристичний аналіз вмісту файлів (тільки хеш-порівняння);
 - Інтеграція з централізованими системами керування (SIEM,

EDR);

- Мобільна версія для Android або iOS;
- Міжплатформна портативність (Linux, macOS).
- Технічні обмеження:
 - Програма не може запуститися на пошкоджених або заражених системах, де критичні компоненти ОС вже скомпрометовані;
 - Не гарантується виявлення 100% сучасних руткітів, що мають більш глибоку інтеграцію з ядром, ніж сама програма;
 - Евристичний аналіз базується на статичних ознаках файлу, без використання динамічної пісочниці (sandbox) або машинного навчання.
- Вимоги до персоналу:
 - Адміністратор/розробник повинен розумітися на основах Windows API та архітектури процесів;
 - Для налаштування конфіг-файлів потрібна базова грамотність у роботі з текстовими файлами та реєстром Windows.

2.1.7. Ризики та план мітигації

Таблиця 2.1. Таблиця ризиків

#	Ризик	Ймовірність	Вартість	Мітигація
P1	Затримання доступу до файлів, які використовуються іншими процесами	Середня	Низька	Перехоплення виключення QFile::open() з виведенням попередження; файл пропускається та позначається у звіті як «недоступний»
P2	Хибно-позитивні збіги хешів через колізії MD5	Низька	Середня	Використання додаткової верифікації імені файлу поруч з хешем; документування обмеження у звіті
P3	Переповнення диску	Низька	Середня	Попередня перевірка вільного місця перед

#	Ризик	Ймовірність	Вартість	Мітигація
	файлами карантину			переміщенням; виведення попередження якщо місця менше 100 МБ
P4	Некоректне відображення шляхів процесів (відсутність прав доступу)	Середня	Низька	Обробка ERROR_ACCESS_DENIED при QueryFullProcessImageName(); відображення «N/A» замість краху

2.1.8. Етапи розробки та план проекту (MVP)

Таблиця 2.2. Етапи розробки

Етап	Блок	Завдання	Результат
Етап 1	Налаштування + Ядро	Ініціалізація Qt-проекту, CMakeLists, структура директорій; клас Logger; клас SignatureDB (завантаження signatures.db); клас Scanner (рекурсивний обхід + MD5)	Консольна версія: сканер знаходить EICAR-тестовий файл
Етап 2	GUI + Процеси + Карантин	MainWindow з QTabWidget; вкладка “Сканування” (кнопка, прогрес-бар, таблиця); QThread для сканування; вкладка “Процеси” (Windows API); клас Quarantine + вкладка “Карантин”	Повністю інтерактивний GUI; сканування, перегляд процесів, переміщення до карантину
Етап 3	Завершення + Тестування	Вкладка “Журнал” та “Налаштування” (білий список); генерація звіту .txt (клас ReportWriter); тестування на EICAR; фіксація дефектів; скріншоти для записки	Робоча програма, файл звіту scan_*.txt, звіт про тестування, фінальний коміт

Структура проекту Qt:

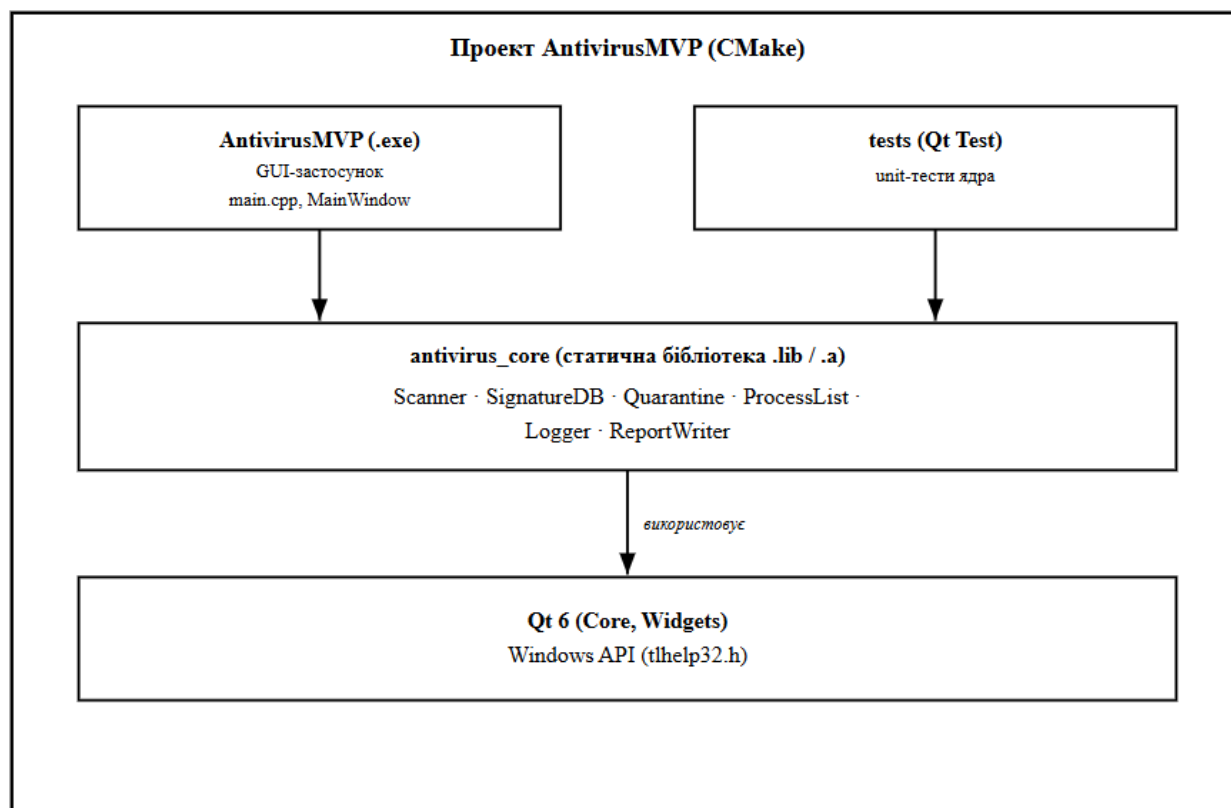


Рис. 2 1. Структурна проекту Qt

Загальна тривалість розробки MVP: 3 етапи

2.2. Структурна схема програми

2.2.1. Опис архітектури системи

Архітектура програми побудована за **модульним принципом**: кожен функціональний блок реалізований як окремий C++-клас з чітко визначеним інтерфейсом і мінімальними залежностями. Взаємодія між модулями відбувається через сигнали та слоти Qt (механізм publisher-subscriber). Весь рівень представлення зосереджений у класі MainWindow, що виступає єдиною точкою входу для дій користувача.

Система складається з двох архітектурних рівнів: - **Низькорівневий рівень (ядро)** - C++ класи Scanner, SignatureDB, Quarantine, ProcessList, Logger, ReportWriter. Компілюються як статична бібліотека antivirus_core.lib, що забезпечує незалежність бізнес-логіки від GUI. - **Рівень представлення (GUI)** - клас MainWindow на Qt Widgets, що підключає бібліотеку ядра та відображає результати.

2.2.2. Структурна схема модулів

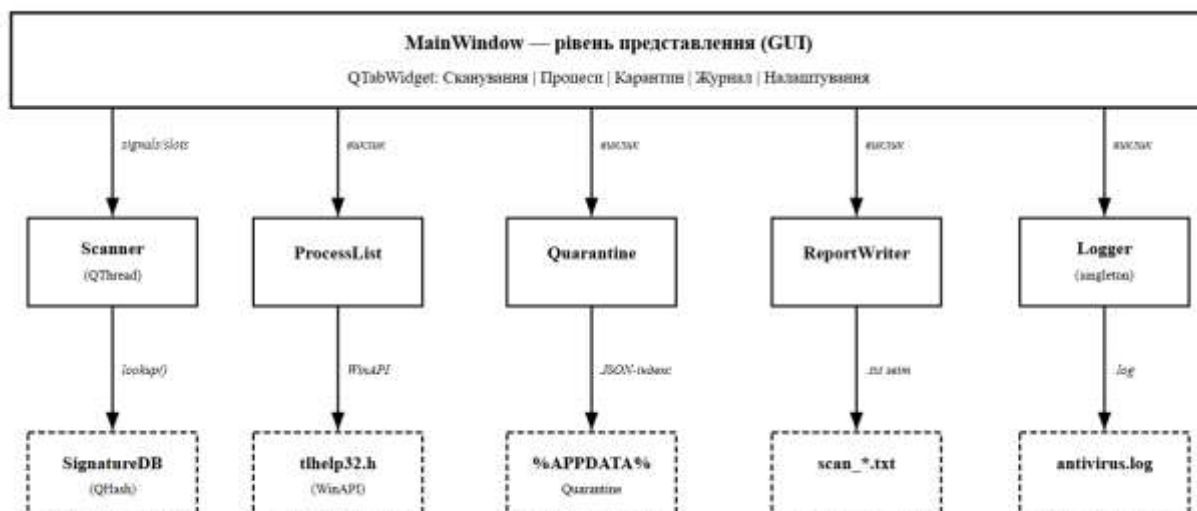


Рис. 2.2. Структурна схема модулів антивірусного сканера

2.2.3. Опис модулів системи

Таблиця 2.3. Структура модулів системи

Модуль	Клас	Відповідальність
Сканер	Scanner	Рекурсивний обхід директорії через QDirIterator; обчислення MD5-хешу кожного файлу через QCryptographicHash; перевірка по SignatureDB та whitelist; генерація сигналу з результатом для GUI
База сигнатур	SignatureDB	Завантаження та парсинг signatures.db; зберігання хешів у QHash<QString, QString> (хеш → назва загрози); метод lookup(hash)
Карантин	Quarantine	Переміщення файлів через QFile::rename(); зберігання метаданих (оригінальний шлях, дата) у quarantine_index.json; відновлення та видалення
Список процесів	ProcessList	Виклик CreateToolhelp32Snapshot() + QueryFullProcessImageName(); повертає QVector<ProcessInfo> з полями: PID, ім'я, шлях, підозрілість
Журнал	Logger	Singleton; запис рядків з часовою міткою та рівнем (INFO/WARNING/ERROR) до файлу через QTextStream; thread-safe через QMutex
Звіти	ReportWriter	Формування текстового файлу зі статистикою сканування та списком загроз; назва файлу з часовою міткою
Головне вікно	MainWindow	Управляє QTabWidget; запускає Scanner у QThread; відображає результати у QTableWidgetItem; з'єднує сигнали ядра зі слотами GUI

2.3. Вибір засобів реалізації

2.3.1. Вибір мови програмування

Для даного курсового проєкту порівнювалися три мови системного рівня:

Таблиця 2.4. Порівняння мов програмування

Критерій	C++ 17	Rust	C# (.NET)
Доступ до Windows API	Нативний (windows.h)	Через крейти (winapi)	P/Invoke (непряме)
Управління пам'яттю	Ручне (RAII)	Власність (borrow checker)	GC (непередбачувані паузи)
Підтримка Qt	Рідна мова Qt	Обмежена (qt-rs)	Відсутня
Швидкість компіляції	Середня	Повільна	Висока
Поріг входження	Середній	Високий	Низький
Зрілість екосистеми	Висока (30+ років)	Середня	Висока

Обрано: C++ 17. Обґрунтування: Qt написано на C++ і надає весь необхідний інструментарій нативно. Нативний доступ до tlhelp32.h без прошарків. RAII та розумні покажчики (std::unique_ptr, std::shared_ptr) забезпечують безпечне управління ресурсами без GC-пауз.

2.3.2. Вибір фреймворку GUI

Таблиця 2.5. Порівняння фреймворків

Критерій	Qt 6 Widgets	wxWidgets	GTK+
Кросплатформеність	Windows/Linux/macOS	Windows/Linux/macOS	Переважно Linux
Якість документації	Відмінна	Середня	Середня
Вбудована підтримка потоків	QThread, QtConcurrent	Потребує зовнішніх libs	Потребує glib
Hashing/Crypto	QCryptographicHash вбудований	Зовнішня lib	Зовнішня lib
Ліцензія	LGPL v3 (комерційна та open-source)	wxWindows License	LGPL
Designer (UI-редактор)	Qt Designer (.ui XML)	Відсутній	Glade

Обрано: Qt 6. Обґрунтування: QCryptographicHash (MD5), QDirIterator, QThread, QFileDialog - всі необхідні компоненти вбудовані без сторонніх залежностей. Qt Designer прискорює розробку GUI. Ліцензія LGPL дозволяє використання у навчальних проєктах.

2.3.3. Інструменти збірки та середовище розробки

- **Збірка:** CMake 3.20+ з find_package(Qt6) - стандарт для C++ проєктів, підтримується VS Code та CLion.
- **IDE:** Qt Creator або VS Code з розширенням CMake Tools та clangd.
- **Компілятор:** MSVC 2022 або MinGW-w64 (GCC 13).

- **Контроль версій:** Git + Gitflow; гілки feature/scanner-core, feature/gui, feature/quarantine.
- **Тестування:** Qt Test (вбудований) для unit-тестів класів SignatureDB та Scanner.

2.4. Проектування

2.4.1. UML-діаграма класів

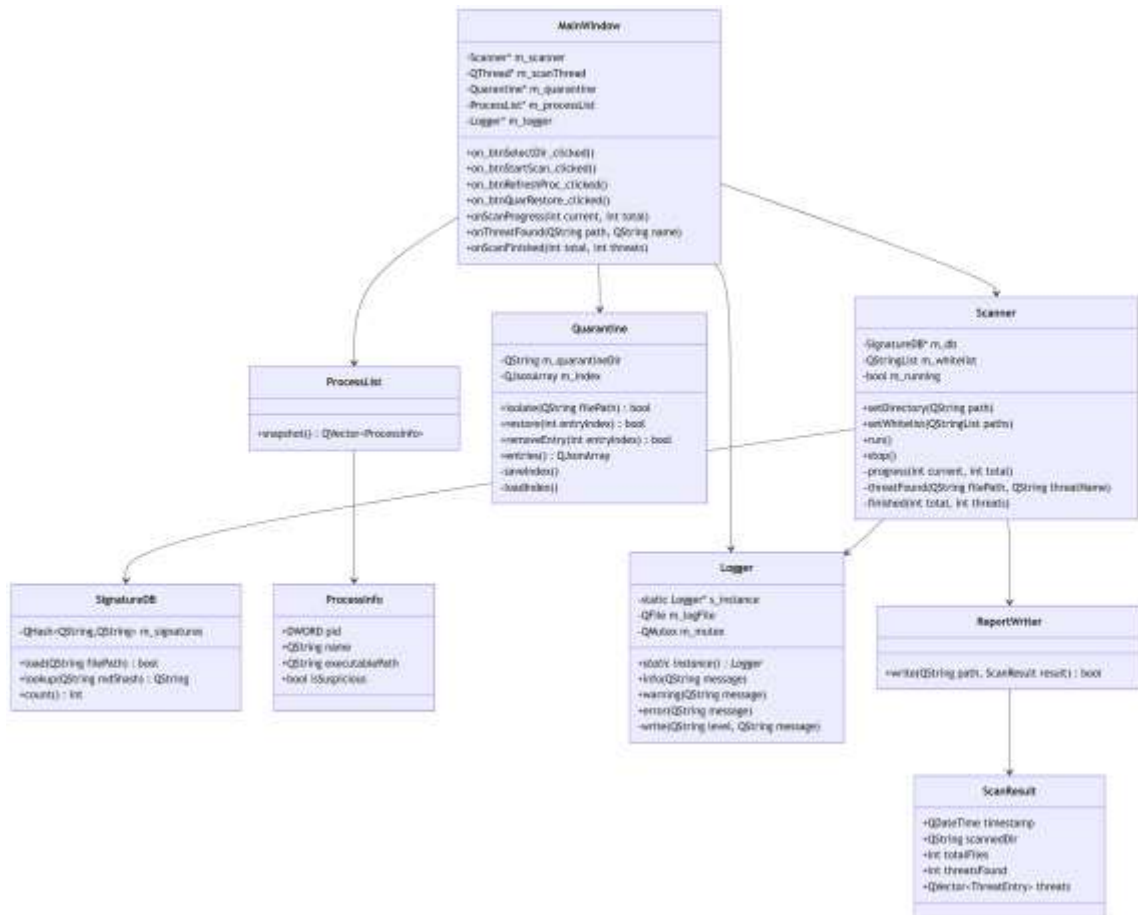


Рис. 2.3. UML-діаграма класів антивірусного сканера

2.4.2. Алгоритм сканування файлової системи

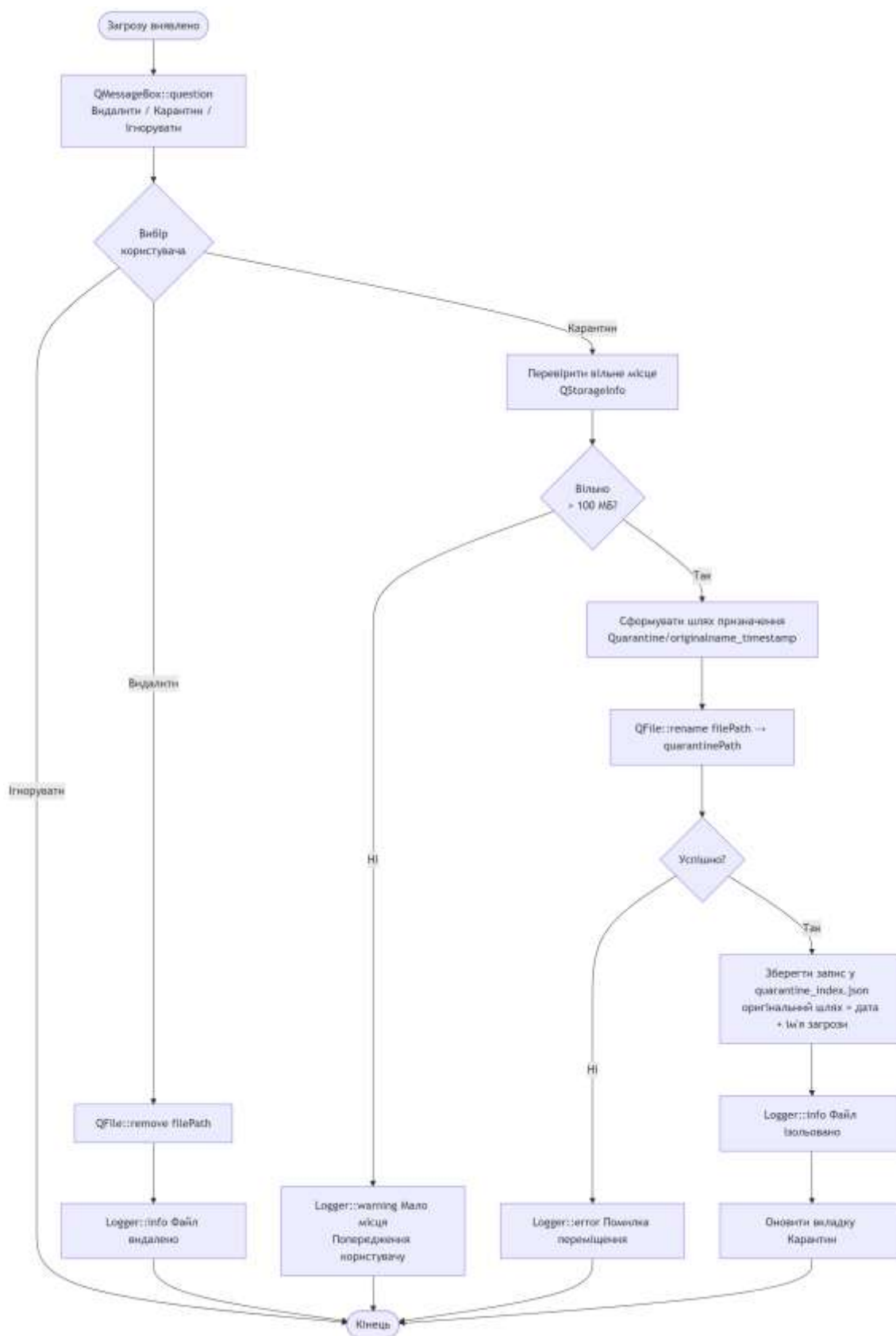


Рис. 2.5. Алгоритм ізоляції виявленої загрози до карантину

2.4.1. Макет інтерфейсу користувача (Wireframe)

Головне вікно програми QMainWindow (800×600 пікселів) містить QTabWidget з п'ятьма вкладками:

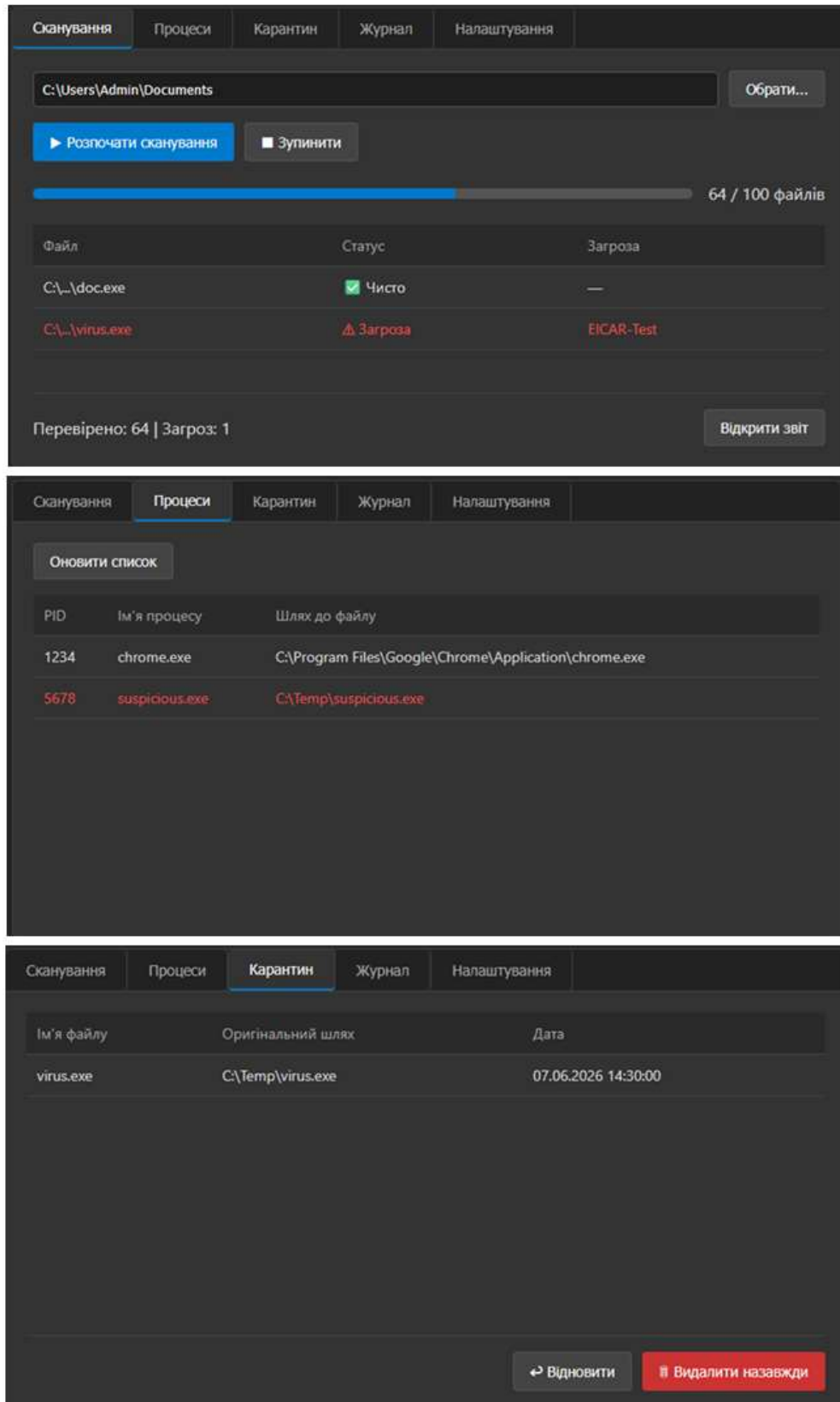


Рис. 2.6. Макети (wireframe) основних вкладок інтерфейсу

3. РЕАЛІЗАЦІЯ

3.1. Планування розробки (User Stories / Scrum)

Відповідно до методичних вказівок, розробку програмного забезпечення організовано за **Scrum**-методологією. Проєкт розбито на **14 User Stories** (US-01–US-14), розподілених по трьох спринтах відповідно до плану розділу 2.1.8. Кожна User Story оформлена як окремий Issue у репозиторії GitHub у форматі «Як [роль], я хочу [дія], щоб [мета]» з переліком критеріїв виконання, лейблами (core, gui, testing, sprint-N) та оцінкою у Story Points (1 SP $\approx$ 0.5 робочого дня). Повний опис всіх Stories збережено у файлі docs/user-stories.md. Розробка велась у гілках feature/* з Merge Request у develop після кожної завершеної Story.

Таблиця 3.1. Зведена таблиця User Stories

Спринт	ID	Назва User Story	SP
1	US-01	Ініціалізація Qt-проєкту та CMake	1
1	US-02	Клас Logger (Singleton, потоко-безпечне журналювання)	1
1	US-03	Клас SignatureDB (завантаження та пошук у базі)	1
1	US-04	Клас Scanner (рекурсивний обхід + MD5 + QThread)	2
1	US-05	Консольна верифікація виявлення EICAR	1
2	US-06	MainWindow + QTabWidget (каркас інтерфейсу)	1
2	US-07	Вкладка «Сканування» з QThread та QProgressBar	2
2	US-08	Клас ProcessList + вкладка «Процеси» (Windows API)	2
2	US-09	Клас Quarantine + вкладка «Карантин»	2
3	US-10	Клас ReportWriter + автозбереження звіту	1
3	US-11	Вкладки «Журнал» та «Налаштування» (whitelist)	1
3	US-12	Unit-тести Qt Test (SignatureDB, Scanner)	1
3	US-13	Інтеграційне тестування та виправлення дефектів	2
3	US-14	Скріншоти для записки та фінальний коміт	1
		Разом	19

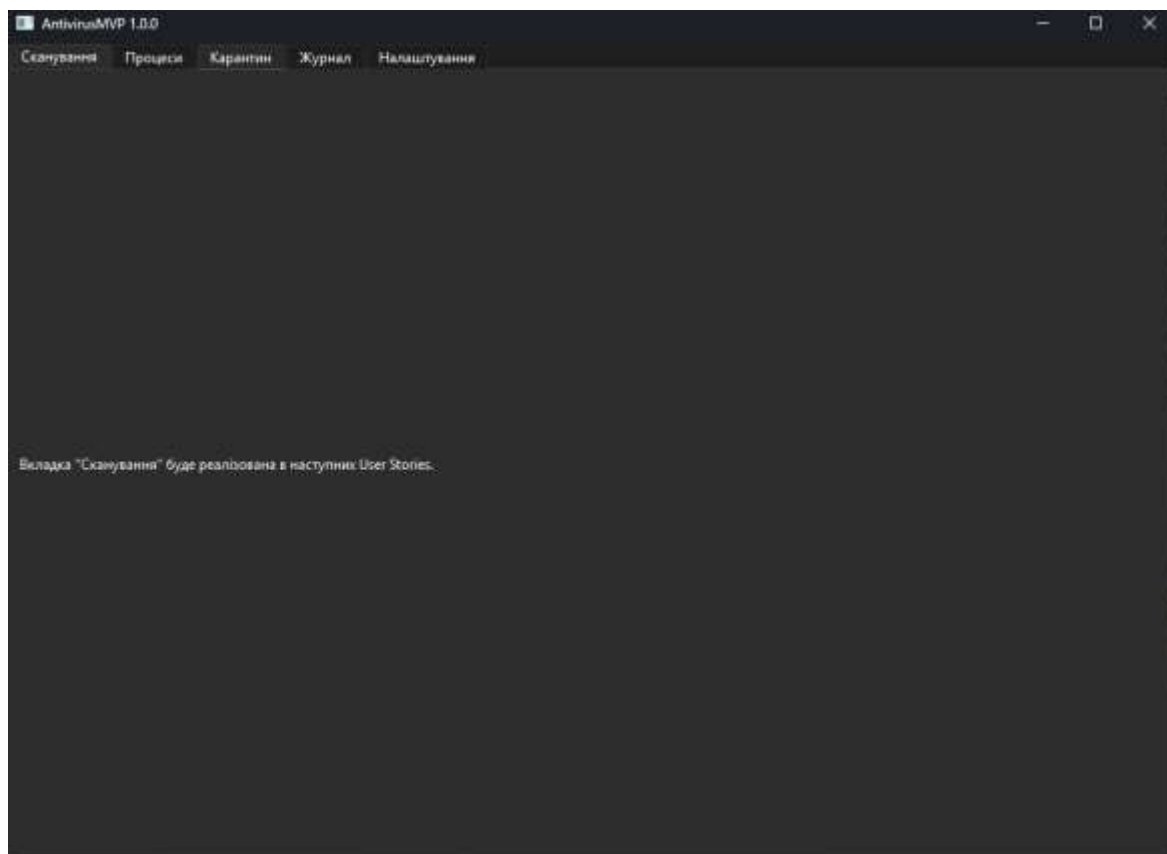


Рис. 3.1. Заглушка (Mock-up) інтерфейсу на ранніх стадіях розробки

3.2. Структура проекту та конфігурація збірки

Проект організовано відповідно до архітектурного рішення розділу 2.2: бізнес-логіка компілюється як статична бібліотека `antivirus_core`, підключена до GUI-виконуваного файлу `AntivirusMVP`. Така структура відповідає вимозі методичних вказівок про наявність низькорівневої складової окремо від рівня представлення.

```
cmake_minimum_required(VERSION 3.20)
project(AntivirusMVP VERSION 1.0.0 LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_AUTOMOC ON)

find_package(Qt6 REQUIRED COMPONENTS Core Gui Widgets)

# Статична бібліотека ядра (низькорівнева складова)
add_library(antivirus_core STATIC
    src/logger.cpp src/signaturedb.cpp src/scanner.cpp
    src/quarantine.cpp src/processlist.cpp src/reportwriter.cpp
)
target_include_directories(antivirus_core PUBLIC src)
target_link_libraries(antivirus_core PUBLIC Qt6::Core psapi)

# GUI-виконуваний файл (рівень представлення)
add_executable(AntivirusMVP WIN32 src/main.cpp src/mainwindow.cpp)
target_link_libraries(AntivirusMVP PRIVATE antivirus_core Qt6::Widgets)
```

3.3. Реалізація ядра системи

3.3.1. Модуль Logger

Клас Logger реалізує Singleton-патерн для централізованого журналювання всіх подій програми. Потоко-безпека (вимога до архітектури QThread) забезпечується QMutex через RAII-обгортку QMutexLocker.

```
// logger.h (фрагмент)
class Logger {
public:
    static Logger* instance();
    bool open(const QString& filePath);
    void info(const QString& msg);
    void warning(const QString& msg);
    void error(const QString& msg);
private:
    static Logger* s_instance;
    QFile      m_logFile;
    QTextStream m_stream;
    QMutex     m_mutex;    // захист від race condition
};

// logger.cpp - метод запису
void Logger::write(const QString& level, const QString& message) {
    QMutexLocker lock(&m_mutex); // RAII-блокування на час запису
    const QString ts = QDateTime::currentDateTime()
        .toString("yyyy-MM-dd HH:mm:ss");
    m_stream << QString("[%1] [%2] %3\n").arg(ts, level, message);
    m_stream.flush();
}
```

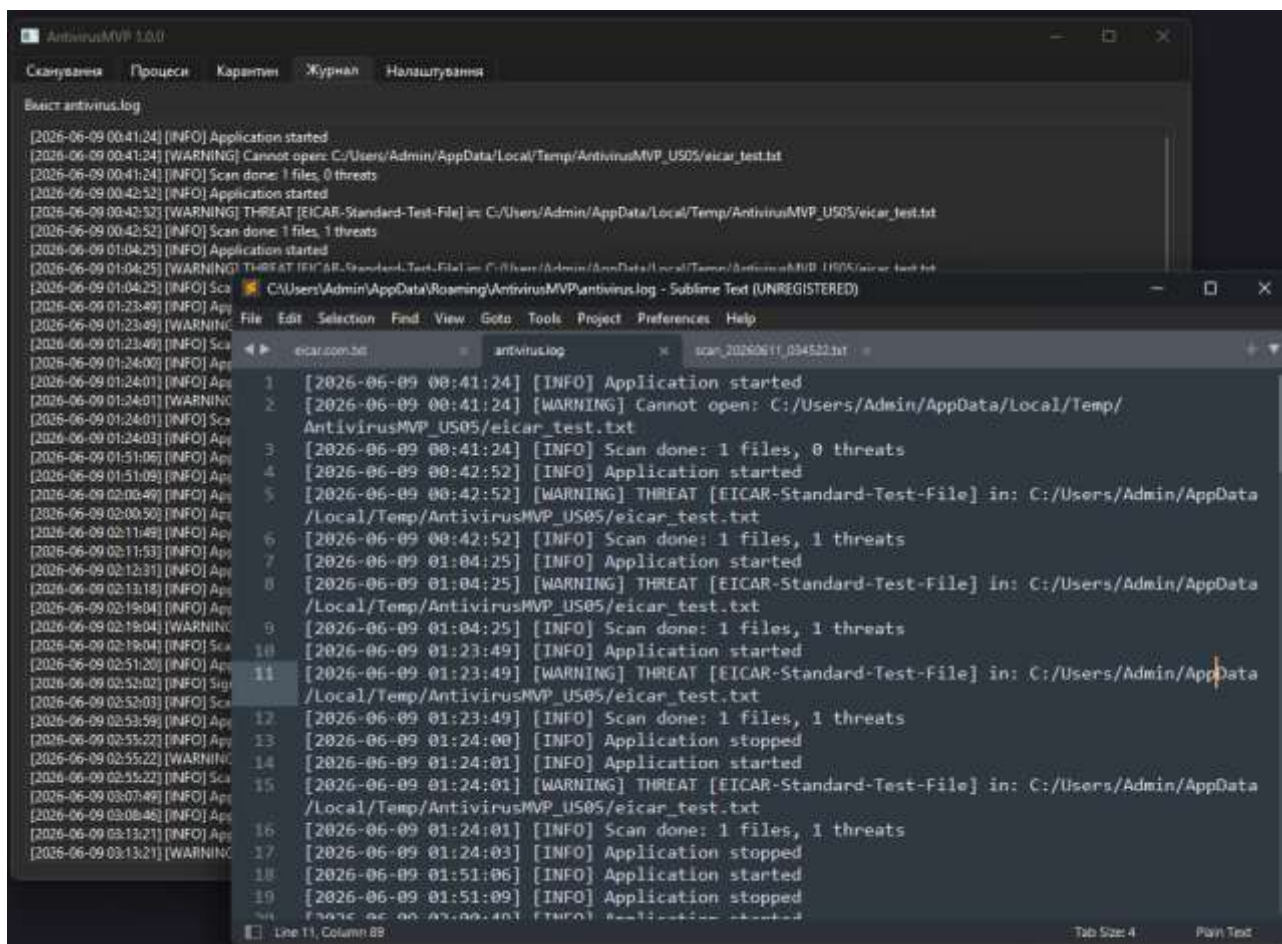


Рис. 3.2. Демонстрація потоко-безпечного запису логів у фізичний файл `antivirus.log`

3.3.2. Модуль SignatureDB

Клас SignatureDB завантажує базу сигнатур у `QHash<QString, QString>` - хеш-таблицю з $O(1)$ -пошуком. Формат файлу `signatures.db`: кожен рядок - `MD5_HEX ThreatName`; рядки, що починаються з `#`, ігноруються як коментарі.

```
// signaturedb.cpp
bool SignatureDB::load(const QString& filePath) {
    QFile file(filePath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text))
        return false;

    static const QRegularExpression ws(R"(\s+)");
    QTextStream in(&file);
    while (!in.atEnd()) {
        const QString line = in.readLine().trimmed();
        if (line.isEmpty() || line.startsWith("#")) continue;
        const QStringList parts = line.split(ws, Qt::SkipEmptyParts);
        if (parts.size() >= 2)
            m_signatures.insert(parts.at(0).toLower(),
                                parts.mid(1).join(' '));
    }
    return !m_signatures.isEmpty();
}

QString SignatureDB::lookup(const QString& md5Hex) const {
    return m_signatures.value(md5Hex.toLower());
}
```

3.3.3. Модуль Scanner

Scanner успадковує QObject для роботи з QThread. Рекурсивний обхід виконується через QDirIterator, MD5-хешування - через QCryptographicHash. Атомарний прапор `std::atomic<bool> m_running` дозволяє безпечно зупинити сканування з GUI-потоків без mutex.

```
// scanner.cpp - метод run() (виконується у QThread)
void Scanner::run() {
    m_running.store(true);

    // Pre-count файлів для точного QProgressBar
    int total = 0;
    QDirIterator counter(m_directory,
        QDir::Files | QDir::NoSymLinks, QDirIterator::Subdirectories);
    while (counter.hasNext()) { counter.next(); ++total; }

    int current = 0, threats = 0;
    QDirIterator it(m_directory,
        QDir::Files | QDir::NoSymLinks, QDirIterator::Subdirectories);

    while (it.hasNext() && m_running.load()) {
        const QString filePath = it.next();
        emit progress(++current, total);

        // Перевірка whitelist (вимога BP5)
        bool skip = false;
        for (const QString& wl : std::as_const(m_whitelist))
            if (filePath.startsWith(wl, Qt::CaseInsensitive))
                { skip = true; break; }
        if (skip) continue;

        const QString md5 = computeMd5(filePath);
        if (md5.isEmpty()) {
            emit logMessage("WARNING", "Cannot open: " + filePath);
            continue;
        }
        if (m_db) {
            const QString threat = m_db->lookup(md5);
            if (!threat.isEmpty()) {
                ++threats;
                emit threatFound(filePath, threat, md5);
            }
        }
    }
    emit finished(current, threats);
}
```

3.3.4. Модуль Quarantine

Quarantine ізолює файл через `QFile::rename()` (атомарна операція в межах одного потоку) до директорії `Quarantine/` у `AppData`. Метадані зберігаються у `quarantine_index.json`. Перед переміщенням перевіряється вільне місце (ризик P3 з T3).

```
// quarantine.cpp - ізоляція файлу
bool Quarantine::isolate(const QString& filePath,
    const QString& threatName) {
```



```
// Перевірка вільного місця (вимога з ризику P3 у T3)
QStorageInfo si(m_dir);
if (si.bytesFree() < 100LL * 1024 * 1024)
    return false;

const QString dest = generateDest(QFileInfo(filePath).fileName());
if (!QFile::rename(filePath, dest)) return false;

QJsonObject obj;
obj["originalPath"] = filePath;
obj["quarantinedPath"] = dest;
obj["threatName"] = threatName;
obj["date"] = QDateTime::currentDateTime()
    .toString("dd.MM.yyyy HH:mm:ss");
m_index.append(obj);
saveIndex();
return true;
}
```

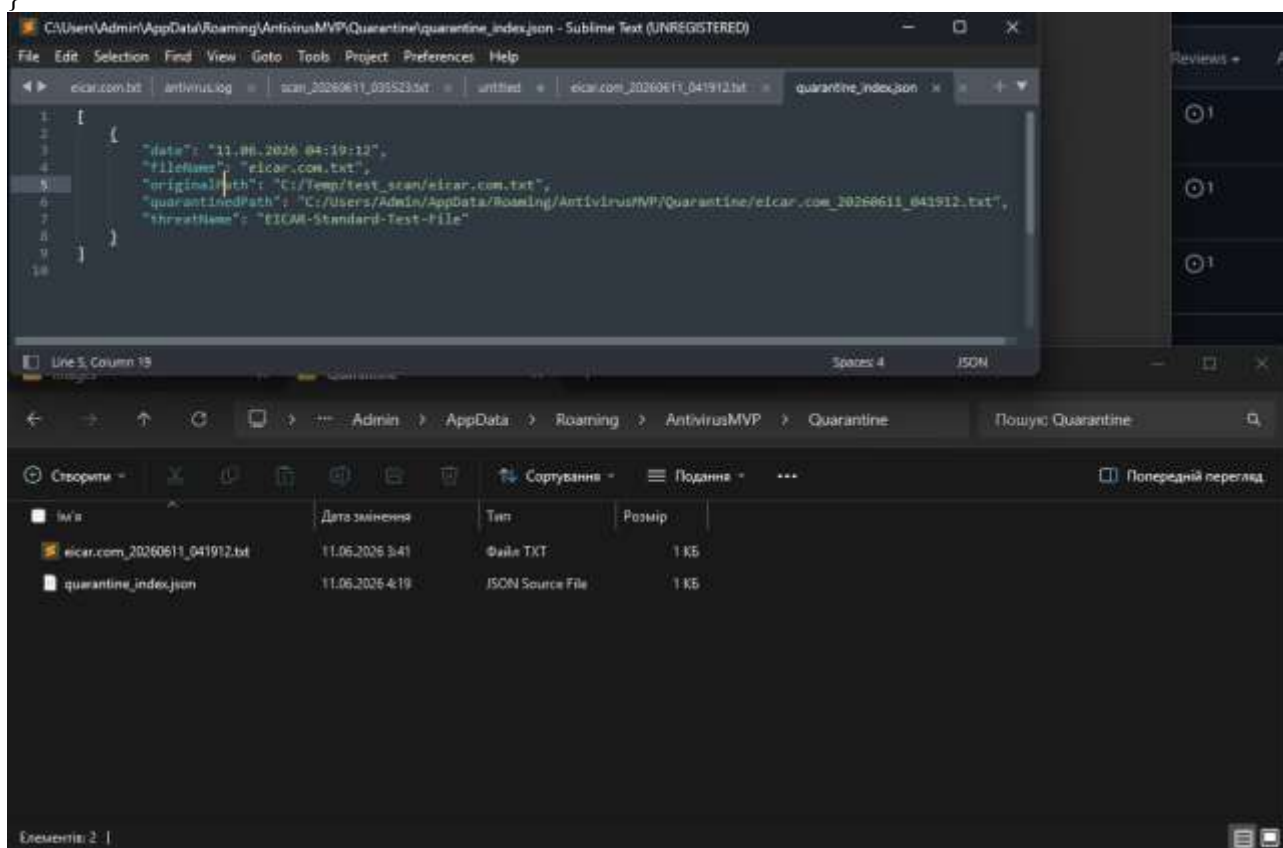


Рис. 3.3. Технічна реалізація збереження метаданих карантину у форматі JSON.

3.3.5. Модуль ProcessList

ProcessList використовує Windows API tlhelp32.h. Для кожного процесу викликається `QueryFullProcessImageNameW()` з прапором `PROCESS_QUERY_LIMITED_INFORMATION`, що дозволяє запит навіть до захищених системних процесів. `ERROR_ACCESS_DENIED` обробляється без краша програми (ризик P4 з T3).

```
// processlist.cpp
QVector<ProcessInfo> ProcessList::snapshot() {
    QVector<ProcessInfo> result;
```

```

HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
if (hSnap == INVALID_HANDLE_VALUE) return result;

PROCESSENTRY32W pe = {}; pe.dwSize = sizeof(pe);
if (Process32FirstW(hSnap, &pe)) {
    do {
        ProcessInfo info;
        info.pid = static_cast<uint32_t>(pe.th32ProcessID);
        info.name = QString::fromWCharArray(pe.szExeFile);

        HANDLE hProc = OpenProcess(
            PROCESS_QUERY_LIMITED_INFORMATION, FALSE, pe.th32ProcessID);
        if (hProc) {
            wchar_t buf[MAX_PATH] = {}; DWORD sz = MAX_PATH;
            if (QueryFullProcessImageNameW(hProc, 0, buf, &sz))
                info.executablePath = QString::fromWCharArray(buf);
            CloseHandle(hProc);
        }
        if (info.executablePath.isEmpty())
            info.executablePath = "N/A"; // ERROR_ACCESS_DENIED
        info.isSuspicious = isSuspiciousPath(info.executablePath);
        result.append(std::move(info));
    } while (Process32NextW(hSnap, &pe));
}
CloseHandle(hSnap);
return result;
}

```

	PID	Ім'я	Шлях
1	0	[System Process]	N/A
2	4	System	N/A
3	236	Secure System	N/A
4	280	Registry	N/A
5	780	smss.exe	N/A
6	1152	csrss.exe	N/A
7	1304	wininit.exe	N/A
8	1388	services.exe	N/A
9	1448	Lcalso.exe	N/A
10	1460	lsass.exe	N/A
11	1616	svchost.exe	N/A
12	1628	fontdrvhost.exe	N/A
13	1744	WUDFHost.exe	N/A
14	1804	svchost.exe	N/A
15	1860	svchost.exe	N/A
16	1888	WUDFHost.exe	N/A
17	1792	svchost.exe	N/A
18	1688	svchost.exe	N/A
19	2080	svchost.exe	N/A
20	2112	svchost.exe	N/A

Рис. 3.4. Відображення системних процесів та обробка відсутності доступу до їхніх шляхів (N/A).

3.3.6. Модуль ReportWriter

ReportWriter формує текстовий звіт зі статистикою та списком загроз у файл scan_YYYYMMDD_NHMMSS.txt. Метод автоматично створює проміжні директорії через QDir::mkpath().

```
// reportwriter.cpp (фрагмент)
bool ReportWriter::write(const QString& reportPath,
                        const ScanResult& result) {
    QDir().mkpath(QFileInfo(reportPath).absolutePath());
    QFile file(reportPath);
    if (!file.open(QIODevice::WriteOnly | QIODevice::Text))
        return false;

    QTextStream out(&file);
    out << "=====\n"
        << " ANTIVIRUS MVP - ЗВІТ СКАНУВАННЯ\n"
        << "=====\n"
        << "Дата/Час: "
        << result.timestamp.toString("dd.MM.yyyy HH:mm:ss") << "\n"
        << "Директорія: " << result.scannedDir << "\n"
        << "Файлів: " << result.totalFiles << "\n"
        << "Загроз: " << result.threatsFound << "\n"
        << "-----\n";
    for (const ThreatEntry& t : result.threats)
        out << "\n [!]" << t.threatName << "\n"
            << " " << t.filePath << "\n"
            << " MD5: " << t.md5 << "\n";
    out << "=====\n";
    return true;
}
```

3.4. Реалізація графічного інтерфейсу

Клас MainWindow будує весь UI програматично (без .ui-файлу) у методах buildScanTab(), buildProcessTab() тощо. Ключовий момент реалізації - інтеграція Scanner з QThread без блокування GUI-потoku. Qt автоматично встановлює Qt::QueuedConnection для сигналів між об'єктами різних потоків, що гарантує безпечне оновлення віджетів тільки з GUI-потoku.

```
// mainwindow.cpp - запуск Scanner у QThread
void MainWindow::startScan() {
    m_scanThread = new QThread(this);
    m_scanner = new Scanner; // без батька - буде moveToThread
    m_scanner->setDirectory(m_dirEdit->text().trimmed());
    m_scanner->setWhitelist(loadWhitelist());
    m_scanner->setSignatureDB(&m_sigDB);
    m_scanner->moveToThread(m_scanThread);

    // Qt::QueuedConnection встановлюється автоматично (cross-thread)
    connect(m_scanThread, &QThread::started, m_scanner, &Scanner::run);
    connect(m_scanner, &Scanner::progress, this, &MainWindow::onScanProgress);
    connect(m_scanner, &Scanner::threatFound, this, &MainWindow::onThreatFound);
    connect(m_scanner, &Scanner::finished, this, &MainWindow::onScanFinished);

    // Автоматичне очищення після завершення потоку
    connect(m_scanner, &Scanner::finished, m_scanThread, &QThread::quit);
}
```

```

connect(m_scanThread, &QThread::finished, m_scanner, &QObject::deleteLater);
connect(m_scanThread, &QThread::finished, m_scanThread, &QObject::deleteLater);

m_scanThread->start();
}

```

При виявленні загрози слот `onThreatFound()` виконується в GUI-поточці (через `QueuedConnection`): додає рядок до таблиці та пропонує користувачу помістити файл до карантину через `QMessageBox::question`.

3.5. База сигнатур та демонстраційний файл EICAR

Для тестування системи використовується стандартний EICAR-тест-файл (European Institute for Computer Anti-Virus Research). Його MD5-хеш є публічно відомим та включений до `signatures.db`:

#	signatures.db	-	формат:	MD5_HEX	ThreatName
44d88612fea8a8f36de82e1278abb02f	EICAR-Standard-Test-File				

EICAR-тестовий рядок (абсолютно нешкідливий, стандартизований):

X5O!P%@AP[4\PZX54(P^7CC)7}\$EICAR-STANDARD-ANTIVIRUS-
TEST-FILE!\$H+H*

Збереження цього рядку у файл `.txt` або `.com` і запуск сканування тієї ж директорії дозволяє перевірити правильність виявлення без використання реального шкідливого коду.

4. ТЕСТУВАННЯ НА ІНСТРКЦІЯ КОРИСТУВАЧА

4.1. Стратегія тестування

4.1.1. Мета та підхід

Метою тестування є підтвердження відповідності розробленого програмного забезпечення функціональним вимогам ВР1–ВР7 та нефункціональним вимогам НВ1–НВ10, визначеним у попередніх розділах пояснювальної записки.

Згідно з тест-планом (test-cases.md), застосовано трирівневу стратегію тестування:

- **Модульне тестування (Unit-testing)** — проводиться з використанням фреймворку Qt Test для перевірки окремих компонентів системи (SignatureDB, Scanner, Quarantine). Метою є верифікація коректності бізнес-логіки незалежно від графічного інтерфейсу.
- **Інтеграційне тестування (Integration testing)** — виконується шляхом ручного тестування графічного інтерфейсу користувача відповідно до тест-кейсів ТС-01–ТС-12. Перевіряється взаємодія між основними підсистемами (сканування, карантин, GUI, журналювання, обробка процесів).
- **Регресійне тестування (Regression testing)** — здійснюється після внесення змін та виправлення дефектів шляхом повторного виконання релевантних тест-кейсів (зокрема ТС-04, ТС-05, ТС-08).

Такий підхід дозволяє забезпечити повне покриття як логіки роботи системи, так і її інтеграції в умовах реального використання.

4.1.2. Тестове середовище

Тестування проводилося у наступному програмно-апаратному середовищі:

Таблиця 4.1. Тестове середовище

Параметр	Значення
Операційна система	Windows 11 22H2 (build 22621)
Компілятор	MSVC 2022 (v19.38)
Фреймворк	Qt 6.6.x
Система збірки	CMake 3.28

Параметр	Значення
Версія збірки	AntivirusMVP v1.0.0

4.1.3. Тестові дані

Для перевірки функціональності використовувалися репрезентативні тестові дані:

- **EICAR-тестовий файл** (eicar.com.txt) — стандартний тестовий рядок антивірусних систем, що дозволяє перевірити механізм виявлення загроз. Відповідний MD5-хеш присутній у базі сигнатур (*signatures.db*).
- **Тестові директорії:**
 - порожня директорія — для перевірки граничних умов;
 - директорія з великою кількістю файлів — для перевірки продуктивності та реакції GUI;
- **Білий список (whitelist.txt)** — містить шляхи, які повинні виключатися зі сканування;
- **Заблокований файл (locked.dat)** — використовується для перевірки обробки помилок доступу (ризик P1).

4.1.4. Критерії успішності тестування

Тестування вважається успішним за виконання наступних умов:

- всі критичні тест-кейси (ТС-01, ТС-04, ТС-11) виконані успішно;
- всі тест-кейси ТС-01–ТС-12 пройдені без дефектів, що блокують функціональність;
- критичні та високопріоритетні дефекти виправлені;
- дефекти середнього та низького рівнів задокументовані як відомі обмеження системи.

4.2. Результати виконання тест-кейсів

Повний опис тест-кейсів наведено у файлі test-cases.md. Узагальнені результати тестування подано у таблиці 4.2.

Таблиця 4.2. Результати тестування функціональних вимог

ID	Вимога	Назва тест-кейсу	Результат
ТС-01	BP1	Сканування з виявленням EICAR	Пройдено

ID	Вимога	Назва тест-кейсу	Результат
TC-02	BP1	Сканування порожньої директорії	Пройдено
TC-03	BP1	Зупинка сканування	Пройдено
TC-04	BP2	Ізоляція файлу до карантину	Пройдено
TC-05	BP2	Відновлення файлу з карантину	Пройдено
TC-06	BP3	Відображення списку процесів	Пройдено
TC-07	BP3	Обробка процесів без доступу	Пройдено
TC-08	BP4	Автоматичне збереження звіту	Пройдено
TC-09	BP5	Виключення через білий список	Пройдено
TC-10	BP6	Журнування операцій	Пройдено
TC-11	BP7	Реагування GUI під час сканування	Пройдено
TC-12	HB5	Обробка заблокованого файлу	Пройдено

Результати демонструють повне покриття функціональних вимог та коректну роботу програмного забезпечення у різних сценаріях використання.

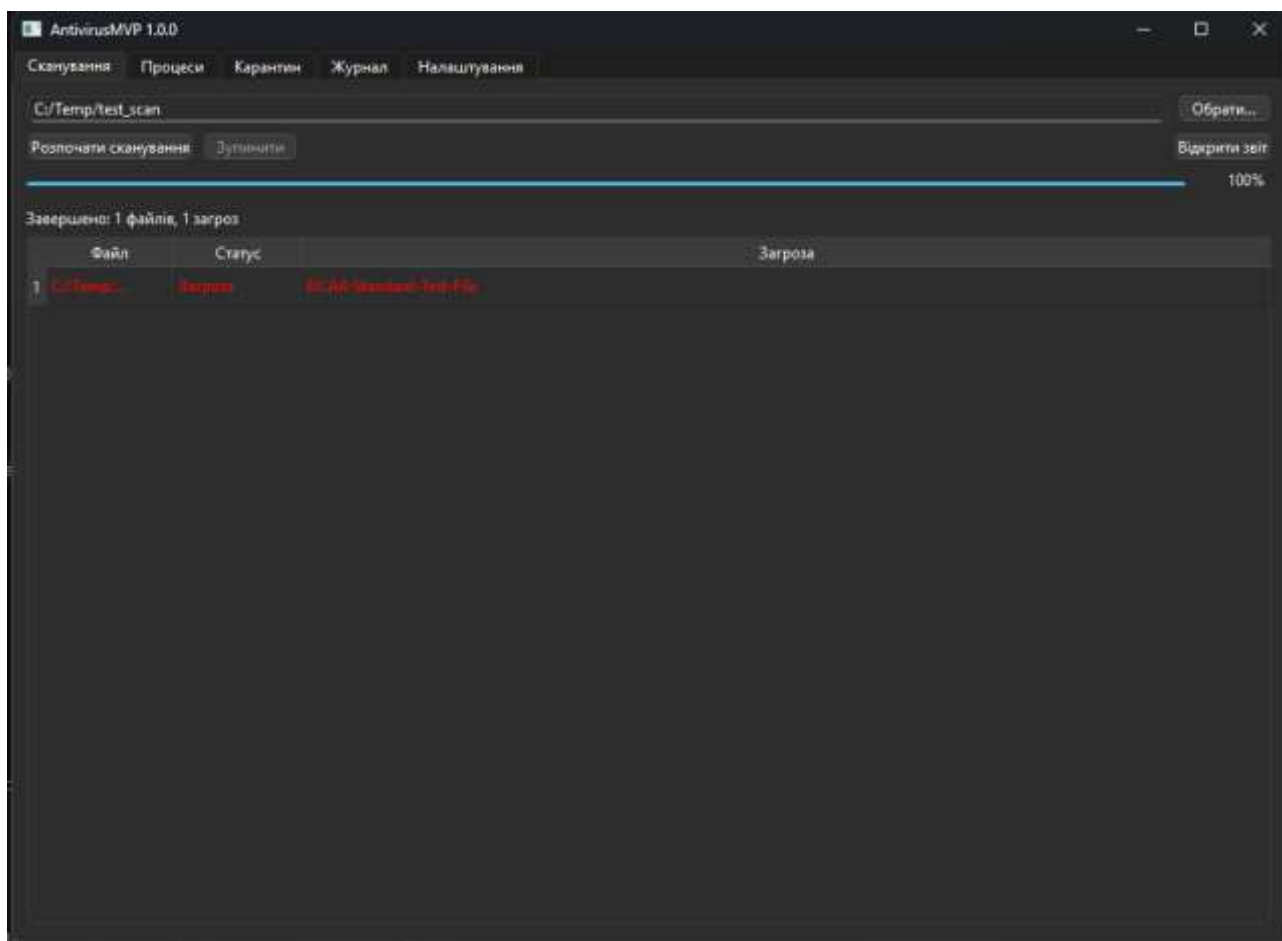


Рис. 4.1. Успішне проходження тест-кейсу TC-01: виявлення загрози EICAR.

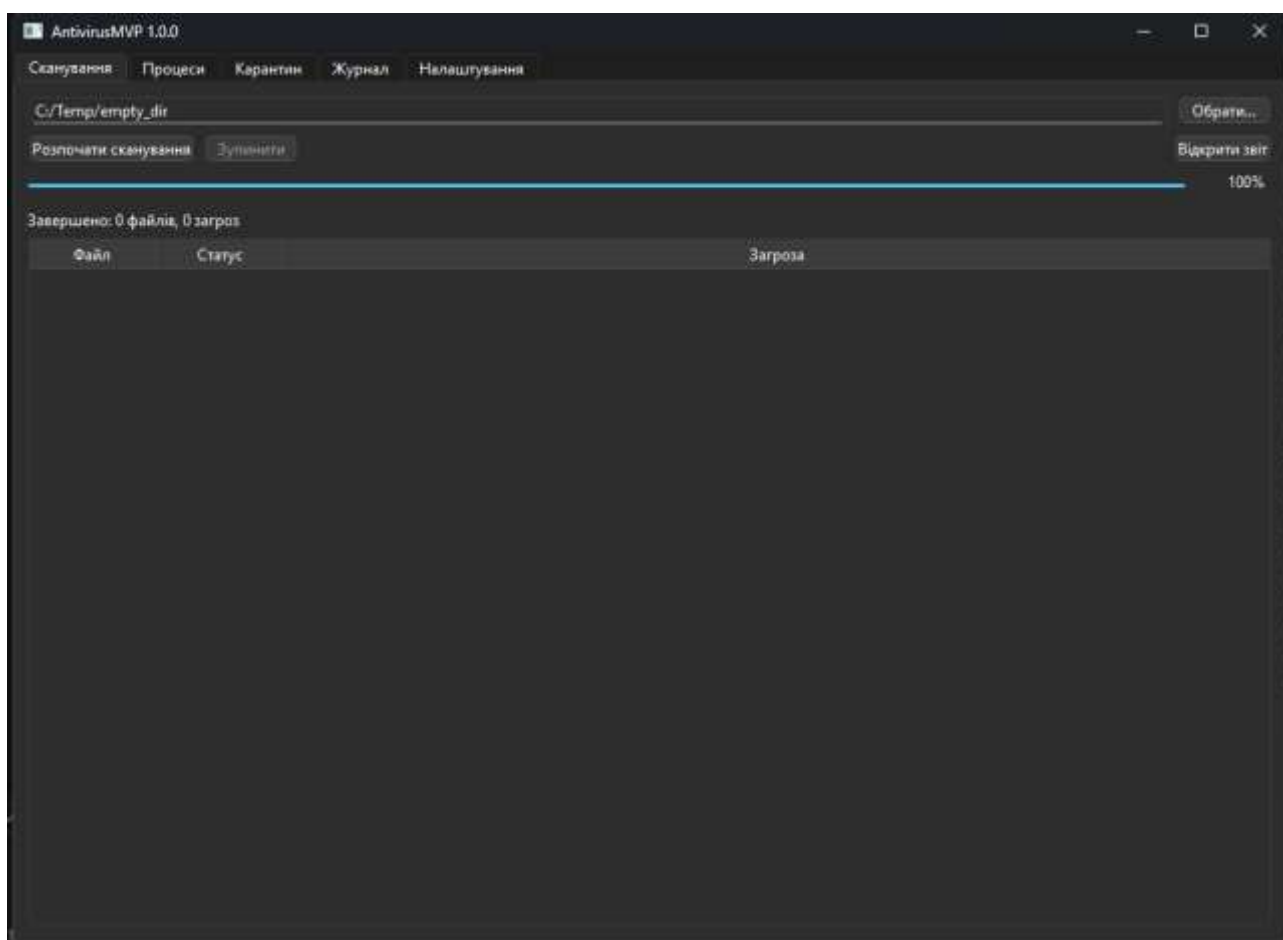


Рис. 4.2. Проходження тест-кейсу TC-02: обробка порожньої директорії.

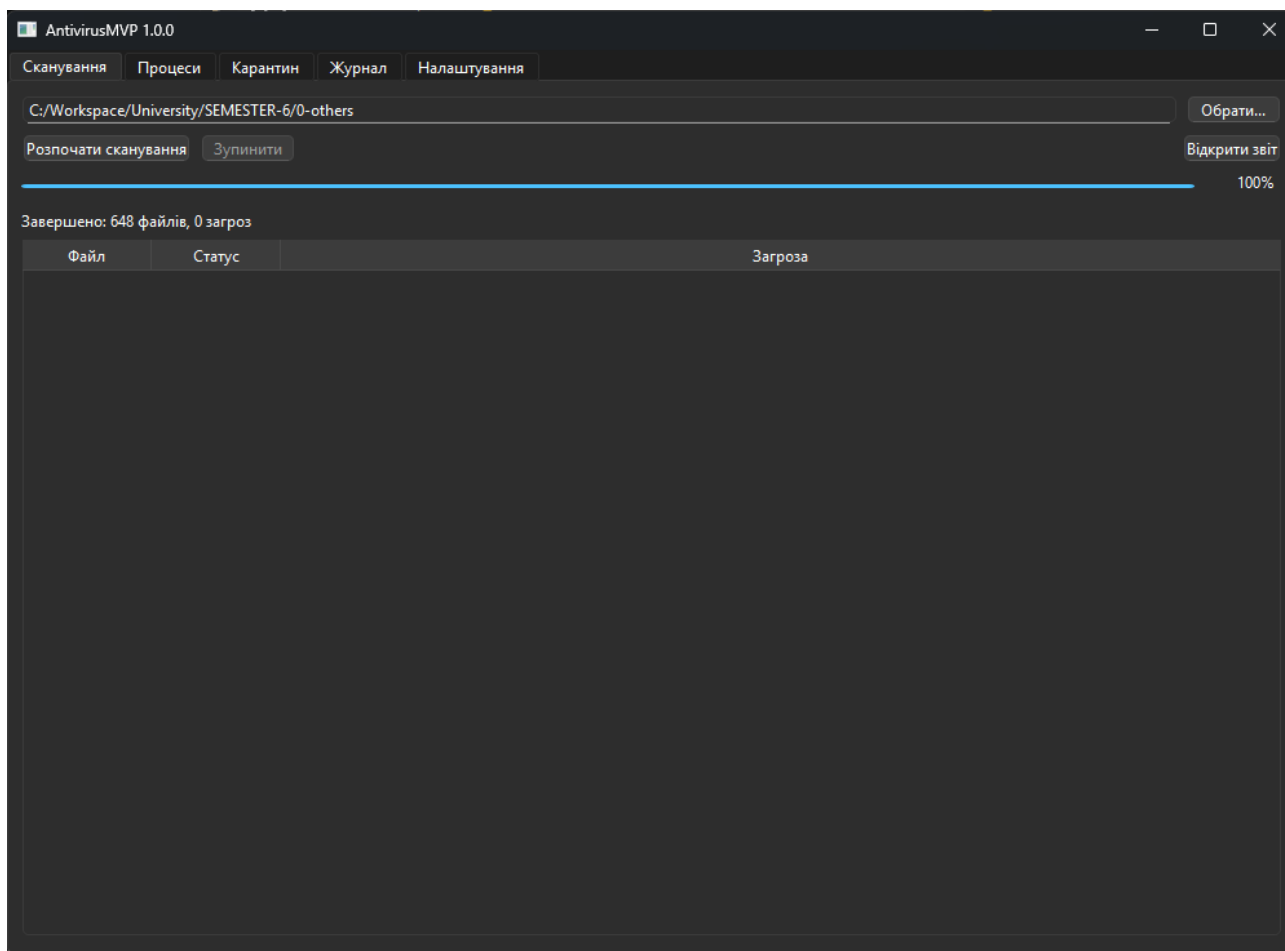


Рис. 4.3. Навантажувальне тестування: сканування директорії з великою кількістю файлів.

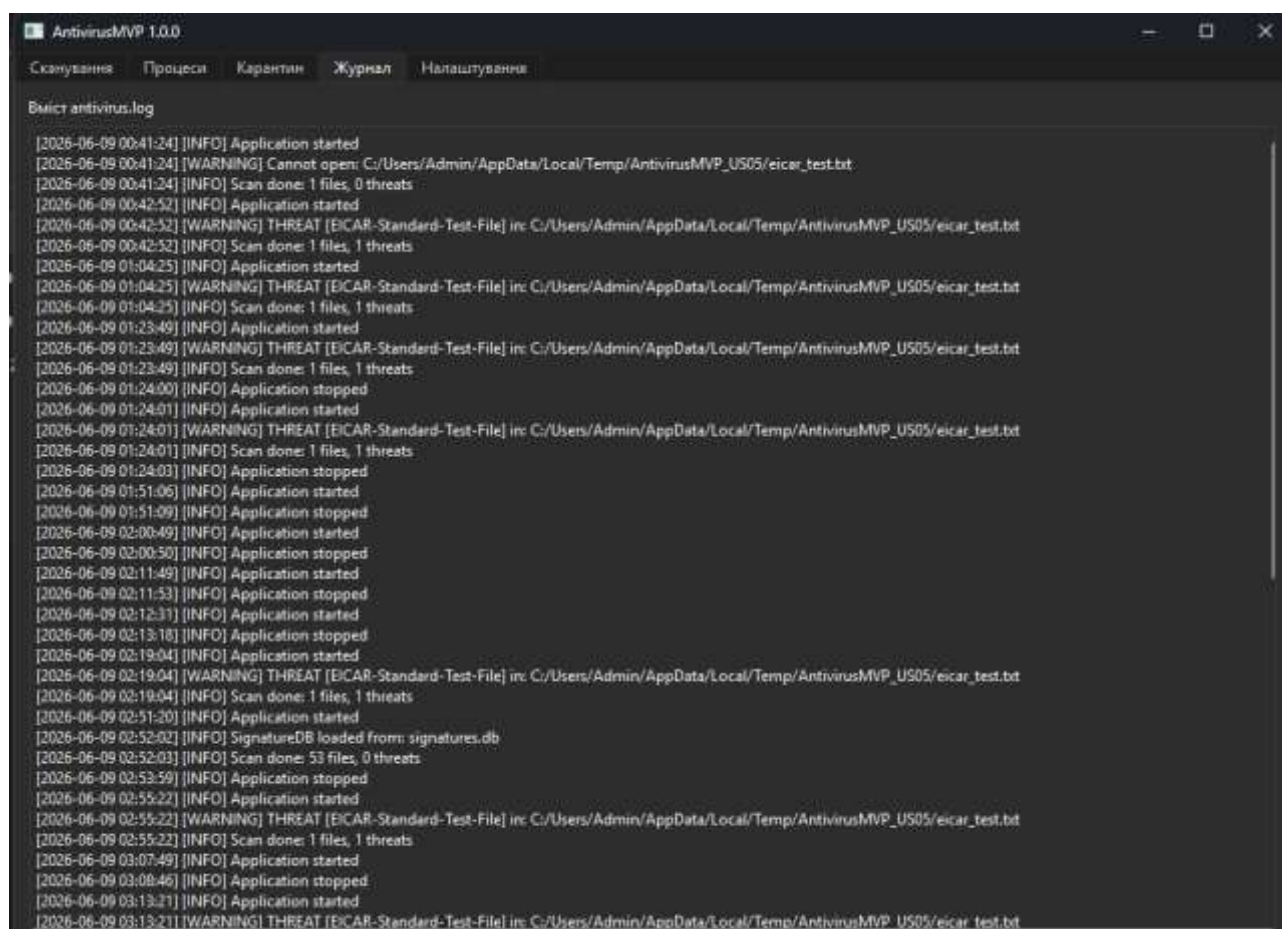


Рис. 4.4. Проходження тест-кейсу TC-10: перегляд журналу подій програми.

4.3. Виявлені дефекти

У процесі тестування виявлено та задокументовано критичного пріоритету.

4.3.1. Дефект BUG-01

Назва: некоректна активація кнопки відкриття звіту

Вимога: ВР4-Звітування

Проблема полягала у тому, що після примусової зупинки сканування кнопка відкриття звіту залишалася неактивною. Причиною було те, що сигнал завершення сканування не генерувався у випадку переривання процесу.

Дефект було виправлено шляхом коректного опрацювання сигналу завершення та підтверджено повторним виконанням тест-кейсу TC-08.

4.4. Інструкція користувача

4.4.1. Встановлення та запуск

Для запуску програмного забезпечення необхідно:

1. Розмістити файли AntivirusMVP.exe, signatures.db та бібліотеки Qt в одній директорії.

2. Запустити виконуваний файл.
3. Після першого запуску автоматично створюється робоча директорія користувача з підкаталогами для карантину, звітів та журналів.

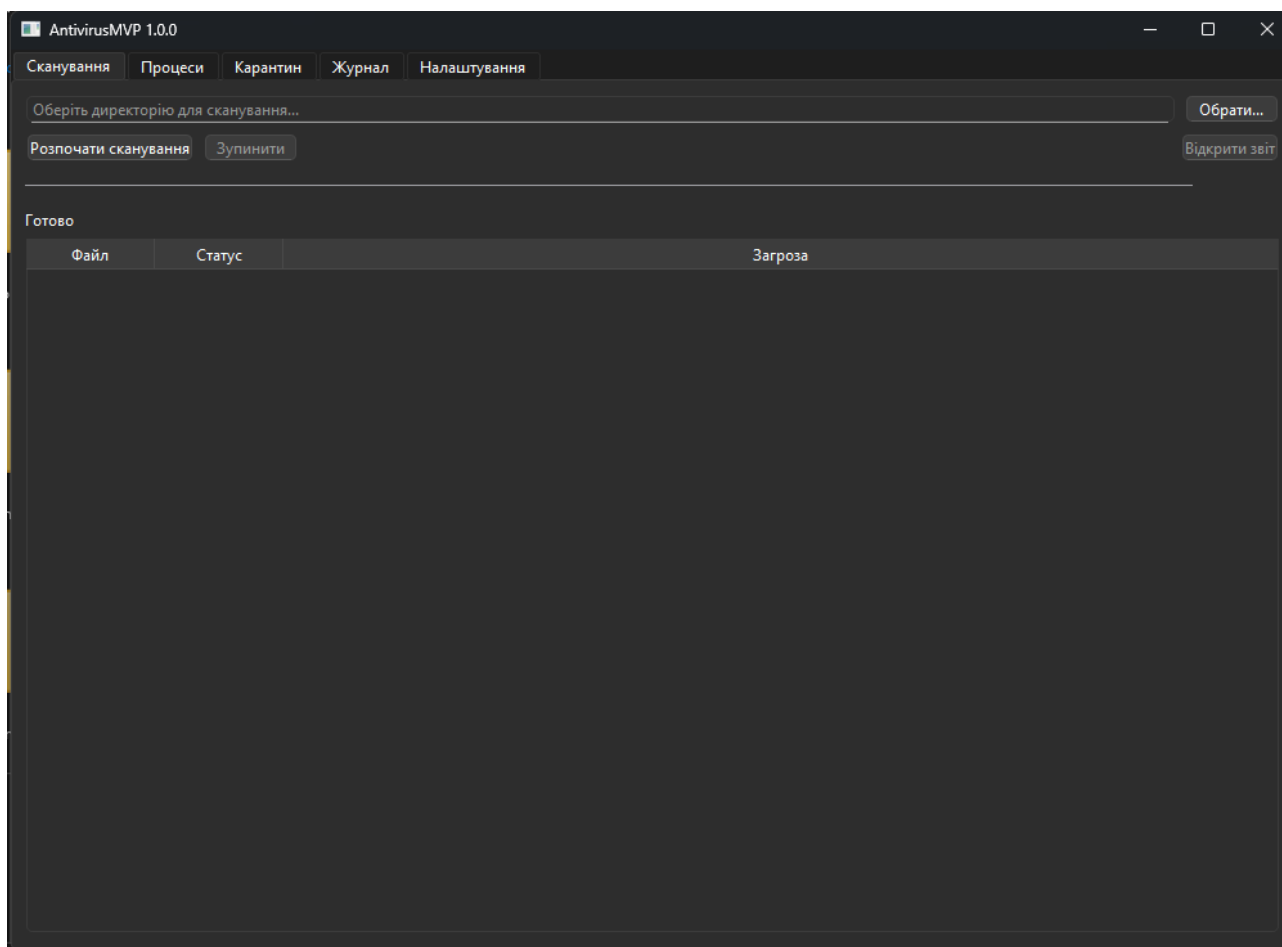


Рис. 4.5. Початковий стан головного вікна програми після запуску.

4.4.2. Виконання сканування

1. Обрати директорію для перевірки.
2. Запустити процес сканування.
3. Дочекатися завершення або зупинити процес за необхідності.
4. У випадку виявлення загроз — обрати дію (ізоляція або ігнорування).
5. Переглянути звіт після завершення.

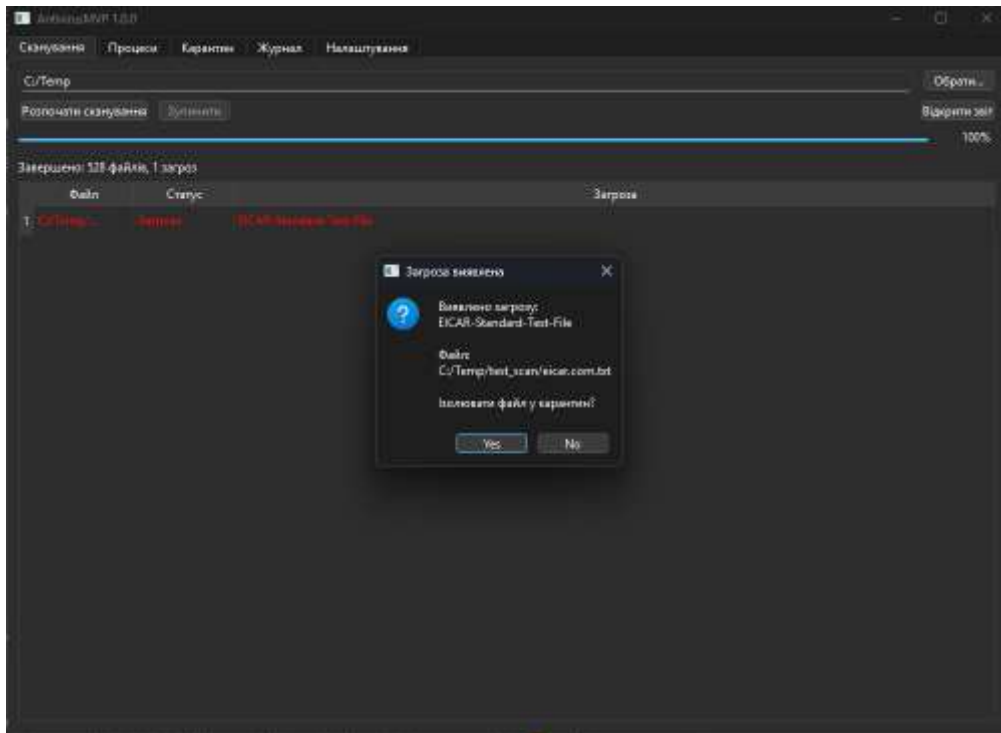


Рис. 4.6. Діалогове вікно попередження про виявлену загрозу.

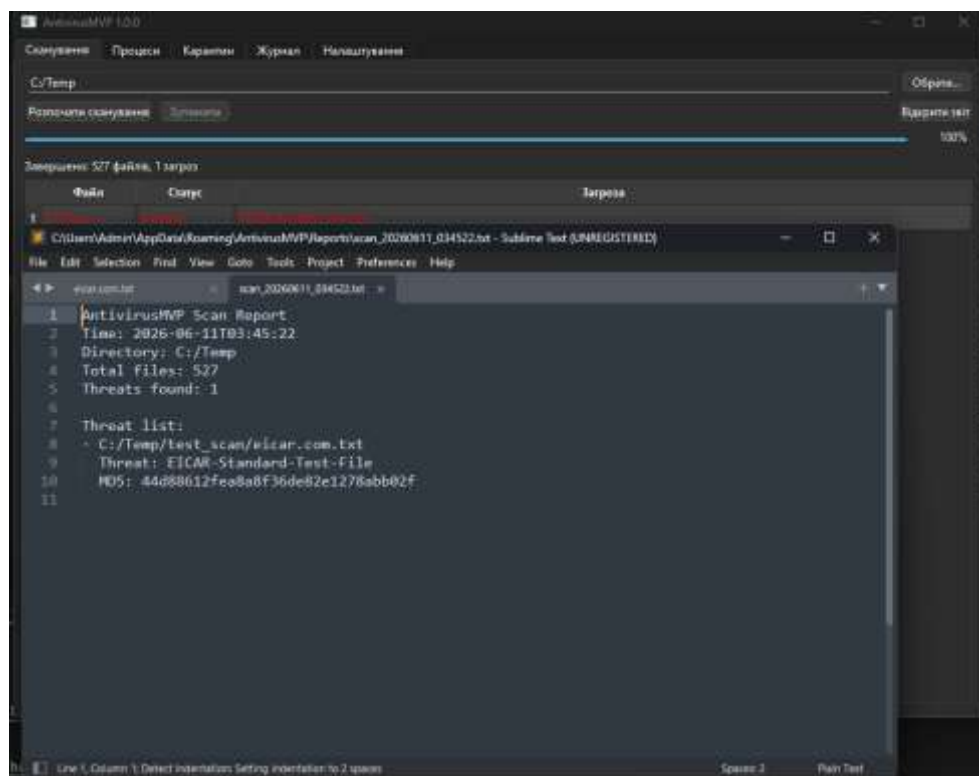


Рис. 4.7. Детальний звіт про сканування, відкритий у текстовому редакторі.

4.4.3. Управління карантинном

Користувач може переглядати, відновлювати або видаляти ізольовані файли через відповідну вкладку інтерфейсу.

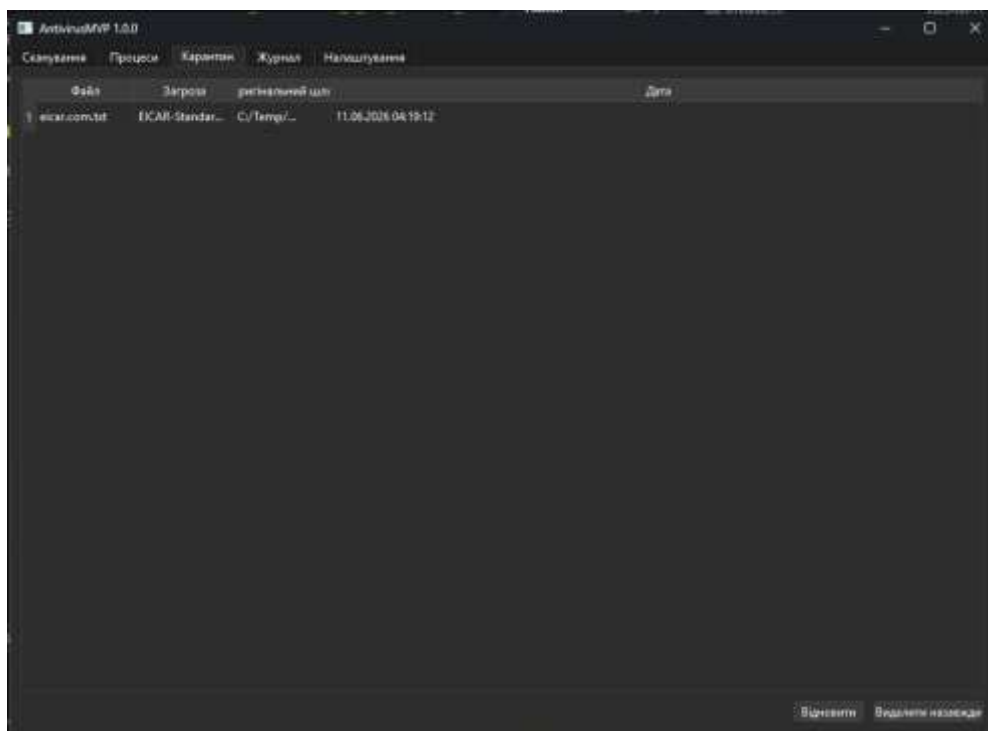


Рис. 4.8. Стан порожнього карантину з успішно ізолюваним шкідливим файлом (знизу).

4.4.4. Перегляд процесів

Вкладка процесів дозволяє отримати інформацію про активні процеси системи, включаючи їхні ідентифікатори та шляхи виконуваних файлів.

AntivirusMVP 1.0.0

СкануванняПроцесиКарантинЖурналНалаштування

Оновити список

	PID	Ім'я	Шлях
256	22572	SearchFilterHos...	N/A
257	22304	SearchFilterHos...	N/A
258	1380	sublime_text.exe	C:\Program Files\Sublime Text\sublime_text.exe
259	444	crash_handler.exe	C:\Program Files\Sublime Text\crash_handler.exe
260	26444	plugin_host-3.3...	C:\Program Files\Sublime Text\plugin_host-3.3.exe
261	14896	plugin_host-3.8...	C:\Program Files\Sublime Text\plugin_host-3.8.exe
262	15780	vctip.exe	C:\Program Files\Microsoft Visual Studio\18\Community\VC\Tools\MSVC\14.44.35207\bin\HostX64\x64\vctip.exe
263	2780	cpptools-srv.exe	C:\Users\Admin\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\bin\cpptools-srv.exe
264	9352	conhost.exe	C:\Windows\System32\conhost.exe
265	21512	cpptools-srv.exe	C:\Users\Admin\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\bin\cpptools-srv.exe
266	22420	conhost.exe	C:\Windows\System32\conhost.exe
267	26144	zen.exe	C:\Program Files\Zen Browser\zen.exe
268	22844	svchost.exe	N/A
269	18000	svchost.exe	N/A
270	26188	zen.exe	C:\Program Files\Zen Browser\zen.exe
271	10844	zen.exe	C:\Program Files\Zen Browser\zen.exe
272	10360	zen.exe	C:\Program Files\Zen Browser\zen.exe
273	20140	smartscreen.exe	C:\Windows\System32\smartscreen.exe
274	2760	7zFM.exe	C:\Program Files\7-Zip\7zFM.exe
275	14344	AntivirusMVP.exe	C:\Users\Admin\Downloads\antivirusmvp\AntivirusMVP.exe

Рис. 4.9. Вкладка перегляду активних процесів із підсвічуванням підозрілих.

4.4.5. Налаштування білого списку

Користувач може додати директорії до білого списку для виключення їх зі сканування.

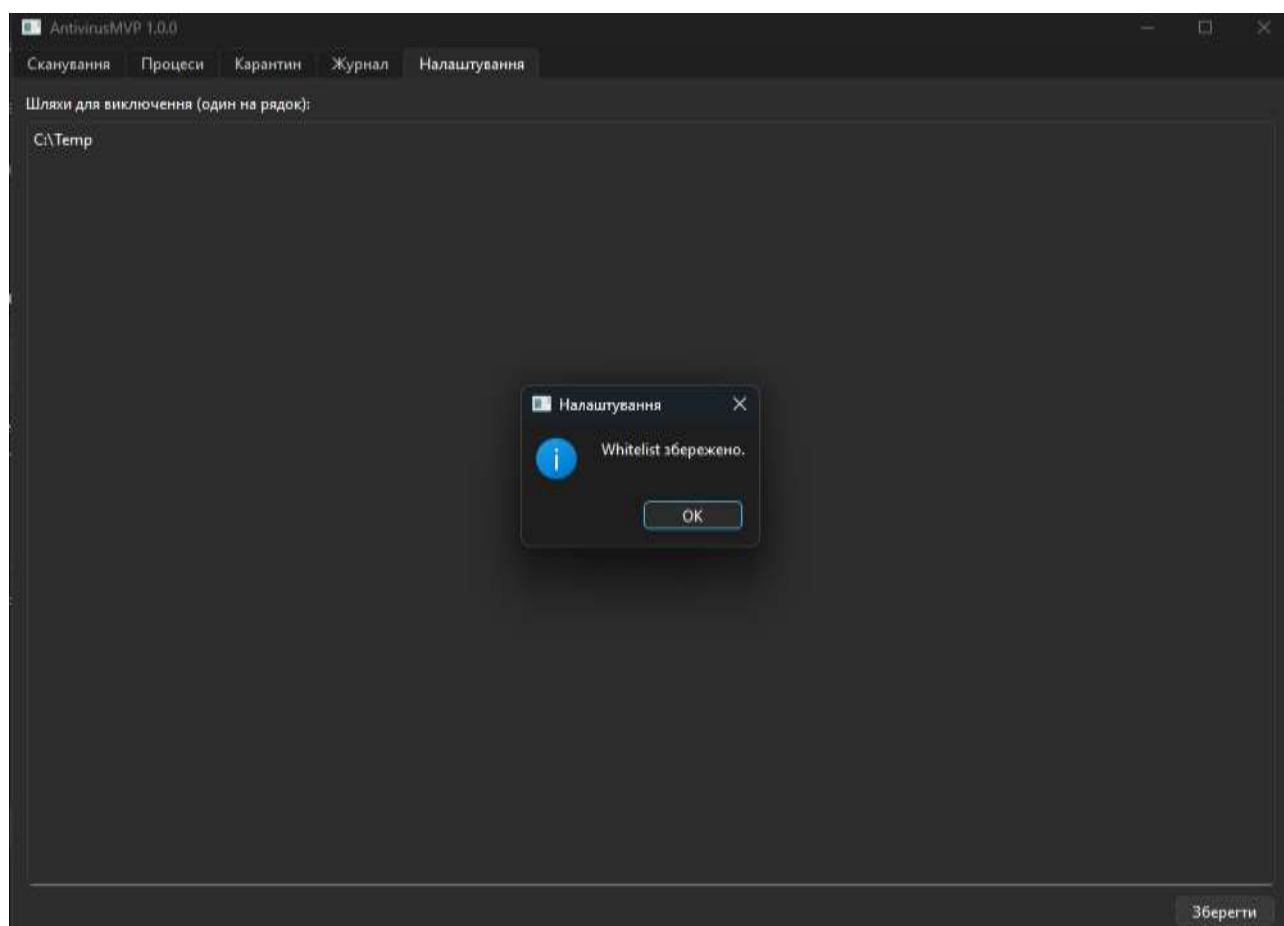


Рис. 4.10. Вкладка перегляду активних процесів із підсвічуванням підозрілих.

4.4.6. Оновлення бази сигнатур

Оновлення бази сигнатур здійснюється шляхом заміни файлу signatures.db.

Підтримується формат запису: “MD5_HEX ThreatName”.

ВИСНОВКИ

У ході виконання курсового проєкту розроблено програмне забезпечення для захисту від вірусів та шкідливих програм - «Antivirus MVP» - відповідно до технічного завдання, сформульованого у розділі 2.1.

Виконані завдання:

1. Розроблено модуль сигнатурного аналізу на базі `QCryptographicHash::Md5` та `QDirIterator`; реалізовано завантаження бази хешів у `QHash<QString,QString>` з $O(1)$ -пошуком.
2. Реалізовано перегляд активних процесів системи через Windows API `CreateToolhelp32Snapshot()` / `QueryFullProcessImageNameW()` з коректною обробкою `ERROR_ACCESS_DENIED`.
3. Розроблено модуль карантину: ізоляція файлів через `QFile::rename()`, збереження метаданих у `quarantine_index.json`, відновлення та видалення.
4. Реалізовано п'ятивкладковий графічний інтерфейс на Qt 6 Widgets (`QMainWindow`, `QTabWidget`); тривале сканування виконується у `QThread`, GUI залишається реагуючим.
5. Розроблено потоко-безпечний Singleton-логер (`QMutex`, `RAII`), генератор текстових звітів та підтримку білого списку виключень.
6. Підготовлено демонстраційну базу сигнатур із EICAR-тестовим файлом.

Розробку організовано за Scrum-методологією: 14 User Stories, 3 спринти, 19 Story Points. Управління версіями - Git Gitflow, репозиторій опублікований на GitHub.

Архітектурні рішення, структура класів та логіка алгоритмів розроблені автором самостійно. GitHub Copilot (модель Claude Sonnet 4.6) використовувався як асистент: генерація шаблонного коду Qt (підключення сигналів/слотів, компонування віджетів), шаблонів документації (User Stories, тест-кейси) та рефакторинг фрагментів коду. Весь вміст пояснювальної записки, технічне завдання та архітектурні рішення опрацьовано та перевірено автором.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Qt Documentation. Qt 6.6 Reference Documentation. - The Qt Company, 2023. - URL: <https://doc.qt.io/qt-6/> (дата звернення: 07.06.2026).
2. Qt Documentation. QCryptographicHash Class. - The Qt Company, 2023. - URL: <https://doc.qt.io/qt-6/qcryptographichash.html> (дата звернення: 07.06.2026).
3. Microsoft. CreateToolhelp32Snapshot function (TlHelp32.h). - Microsoft Learn, 2024. - URL: <https://learn.microsoft.com/en-us/windows/win32/api/tlhelp32/nf-tlhelp32-createtoolhelp32snapshot> (дата звернення: 07.06.2026).
4. EICAR - European Institute for Computer Antivirus Research. Anti-Malware Testfile. - EICAR, 2003. - URL: <https://www.eicar.org/download-anti-malware-testfile/> (дата звернення: 07.06.2026).
5. ClamAV Documentation. ClamAV® open source antivirus engine. - Cisco Systems, 2024. - URL: <https://docs.clamav.net/> (дата звернення: 07.06.2026).
6. Sutherland J., Schwaber K. The Scrum Guide. The Definitive Guide to Scrum: The Rules of the Game. - Scrum.org, 2020. - 13 с. - URL: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Ukrainian.pdf> (дата звернення: 07.06.2026).
7. Stroustrup B. The C++ Programming Language. 4th ed. - Addison-Wesley Professional, 2013. - 1376 с.
8. Atlassian. Gitflow Workflow. - Atlassian Bitbucket Documentation, 2024. - URL: <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow> (дата звернення: 07.06.2026).
9. Microsoft. QueryFullProcessImageNameW function (winbase.h). - Microsoft Learn, 2024. - URL: <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-queryfullprocessimagenamew> (дата звернення: 07.06.2026).
10. ДСТУ 3974-2000. Системи технологій. Правила виконання техніко-економічного обґрунтування. - Держстандарт України, 2000.

ДОДАТКИ

Додаток А. Лістинг програми

Project Structure:

```
.gitignore
CMakeLists.txt
LICENSE
signatures.db
.github/copilot-instructions.md
.github/workflows/pr-build-artifacts.yml
src/logger.cpp
src/logger.h
src/main.cpp
src/mainwindow.cpp
src/mainwindow.h
src/processlist.cpp
src/processlist.h
src/quarantine.cpp
src/quarantine.h
src/reportwriter.cpp
src/reportwriter.h
src/scanner.cpp
src/scanner.h
src/signaturedb.cpp
src/signaturedb.h
tests/test_quarantine.cpp
tests/test_scanner.cpp
tests/test_signaturedb.cpp
```

File Contents:

```
.gitignore:
```

Prerequisites
*.d

docs/

Compiled Object files
*.slo
*.lo
*.o
*.obj

Precompiled Headers
*.gch
*.pch

Linker files
*.ilk

Debugger Files
*.pdb

Compiled Dynamic gitlibraries
*.so
*.dylib
*.dll
.so.

Fortran module files
*.mod
*.smod
```

```

Compiled Static libraries
*.lai
*.la
*.a
*.lib

Executables
*.exe
*.out
*.app

Build directories
build/
Build/
build-*/

CMake generated files
CMakeFiles/
CMakeCache.txt
cmake_install.cmake
Makefile
install_manifest.txt
compile_commands.json

Temporary files
*.tmp
*.log
*.bak
*.swp

vcpkg
vcpkg_installed/

debug information files
*.dwo

test output & cache
Testing/
.cache/

US-05 temporary EICAR file
eicar_test.txt

...

CMakeLists.txt:
```
cmake_minimum_required(VERSION 3.20)
project(AntivirusMVP VERSION 1.0.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)

find_package(Qt6 REQUIRED COMPONENTS Core Gui Widgets Test)

enable_testing()

# —— Core static library (antivirus_core) ——
add_library(antivirus_core STATIC
    src/logger.cpp

```

```

src/logger.h
src/processlist.cpp
src/processlist.h
src/quarantine.cpp
src/quarantine.h
src/reportwriter.cpp
src/reportwriter.h
src/scanner.cpp
src/scanner.h
src/signaturedb.cpp
src/signaturedb.h
)
target_include_directories(antivirus_core PUBLIC src)
target_link_libraries(antivirus_core PUBLIC Qt6::Core)

```

```

if(WIN32)
    target_link_libraries(antivirus_core PUBLIC psapi)
endif()

```

```

# — GUI executable

```

```

add_executable(AntivirusMVP WIN32
    src/main.cpp
    src/mainwindow.cpp
    src/mainwindow.h
)

```

```

target_link_libraries(AntivirusMVP PRIVATE
    antivirus_core
    Qt6::Core
    Qt6::Gui
    Qt6::Widgets
)

```

```

function(add_antivirus_qt_test target_name source_file)
    add_executable(${target_name}
        ${source_file}
    )
    target_link_libraries(${target_name} PRIVATE
        antivirus_core
        Qt6::Core
        Qt6::Test
    )

```

```

if(WIN32)
    # Ensure CTest can run each test binary without missing Qt runtime DLLs.
    add_custom_command(TARGET ${target_name} POST_BUILD
        COMMAND ${CMAKE_COMMAND} -E copy -t ${<TARGET_FILE_DIR:${target_name}>}
        ${<TARGET_RUNTIME_DLLS:${target_name}>}
        COMMAND_EXPAND_LISTS
    )
endif()

```

```

    add_test(NAME ${target_name} COMMAND ${target_name})
endfunction()

```

```

add_antivirus_qt_test(test_signaturedb tests/test_signaturedb.cpp)
add_antivirus_qt_test(test_scanner tests/test_scanner.cpp)
add_antivirus_qt_test(test_quarantine tests/test_quarantine.cpp)

```

```

# Copy signatures.db next to the executable after each build.
add_custom_command(TARGET AntivirusMVP POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different

```

```

    "${CMAKE_SOURCE_DIR}/signatures.db"
    "${<TARGET_FILE_DIR:AntivirusMVP>/signatures.db}"
    COMMENT "Copying signatures.db to output directory"
)

if(WIN32)
    # Deploy Qt runtime DLLs into build/<Config>/ so the app can run directly
    # after building both Debug and Release from VS Code / CMake Tools.
    get_target_property(QT_QMAKE_EXECUTABLE Qt6::qmake IMPORTED_LOCATION)
    get_filename_component(QT_BIN_DIR "${QT_QMAKE_EXECUTABLE}" DIRECTORY)
    find_program(WINDEPLOYQT_EXECUTABLE windeployqt HINTS "${QT_BIN_DIR}")

    if(WINDEPLOYQT_EXECUTABLE)
        add_custom_command(TARGET AntivirusMVP POST_BUILD
            COMMAND "${WINDEPLOYQT_EXECUTABLE}"
                "$<$<CONFIG:Debug>:--debug>$<$<NOT:$<CONFIG:Debug>>:--release>"
                --no-translations
                --no-opengl-sw
                --dir "${<TARGET_FILE_DIR:AntivirusMVP>}"
                "${<TARGET_FILE:AntivirusMVP>}"
            COMMENT "Deploying Qt runtime with windeployqt"
            VERBATIM
        )
    else()
        message(WARNING "windeployqt was not found. AntivirusMVP may fail to run due to missing Qt
DLLs.")
    endif()
endif()

...

signatures.db:
...
# Antivirus MVP — Signature Database
# Format: MD5_HEX ThreatName (whitespace-separated, case-insensitive hash)
# Version: 1.0 (2026-06-07)
# Reference: https://www.eicar.org/download-anti-malware-testfile/

44d88612fea8a8f36de82e1278abb02f EICAR-Standard-Test-File

...

pr-build-artifacts.yml:
...
name: PR Build Artifacts

on:
  pull_request:
    branches:
      - develop
      - main
  workflow_dispatch:

jobs:
  build-windows:
    name: Build Windows Release Artifact
    runs-on: windows-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Install Qt 6

```

```

uses: jurplel/install-qt-action@v4
with:
  version: '6.8.3'
  host: 'windows'
  target: 'desktop'
  arch: 'win64_msvc2022_64'
  cache: true

- name: Configure CMake
  shell: pwsh
  run: >
    cmake -S . -B build
    -G "Visual Studio 17 2022"
    -A x64

- name: Build Release
  shell: pwsh
  run: cmake --build build --config Release

- name: Prepare package folder name
  shell: pwsh
  run: |
    $content = Get-Content CMakeLists.txt -Raw
    $match = [regex]::Match($content, 'project\(\\s*AntivirusMVP\\s+VERSION\\s+([0-9]+\.[0-9]+\.[0-9]+)')
    if ($match.Success) {
      $version = $match.Groups[1].Value
    } else {
      $version = '0.0.0'
    }
    $build = '${{ github.run_number }}'
    $artifactName = "antivirusmvp-$version.$build"
    $packageDir = 'artifact/antivirusmvp'
    "ARTIFACT_NAME=$artifactName" | Out-File -FilePath $env:GITHUB_ENV -Encoding utf8 -Append
    "PACKAGE_DIR=$packageDir" | Out-File -FilePath $env:GITHUB_ENV -Encoding utf8 -Append
    Write-Host "Artifact name: $artifactName"
    Write-Host "Package directory: $packageDir"

- name: Collect and deploy Release artifacts
  shell: pwsh
  run: |
    New-Item -ItemType Directory -Path $env:PACKAGE_DIR -Force | Out-Null
    Copy-Item build/Release/AntivirusMVP.exe $env:PACKAGE_DIR/ -Force
    windeployqt --release $env:PACKAGE_DIR/AntivirusMVP.exe
    Copy-Item signatures.db $env:PACKAGE_DIR/ -Force

- name: Upload build artifacts
  uses: actions/upload-artifact@v4
  with:
    name: ${{ env.ARTIFACT_NAME }}
    path: artifact/
    if-no-files-found: error
...

logger.cpp:
...
#include "logger.h"

#include <QDateTime>
#include <QDir>
#include <QFileInfo>

Logger* Logger::s_instance = nullptr;

```

```

Logger* Logger::instance() {
    if (!s_instance)
        s_instance = new Logger();
    return s_instance;
}

Logger::~Logger() {
    if (m_logFile.isOpen())
        m_logFile.close();
}

bool Logger::open(const QString& filePath) {
    QDir().mkpath(QFileInfo(filePath).absolutePath());
    m_logFile.setFileName(filePath);
    const bool ok = m_logFile.open(QIODevice::Append | QIODevice::Text);
    if (ok) {
        m_stream.setDevice(&m_logFile);
        m_stream.setEncoding(QStringConverter::Utf8);
    }
    return ok;
}

void Logger::write(const QString& level, const QString& message) {
    QMutexLocker lock(&m_mutex);
    const QString ts = QDateTime::currentDateTime()
        .toString(QStringLiteral("yyyy-MM-dd HH:mm:ss"));
    m_stream << QStringLiteral("[%1] [%2] %3\n").arg(ts, level, message);
    m_stream.flush();
}

void Logger::info(const QString& message) { write(QStringLiteral("INFO"), message); }
void Logger::warning(const QString& message) { write(QStringLiteral("WARNING"), message); }
void Logger::error(const QString& message) { write(QStringLiteral("ERROR"), message); }

...

logger.h:
...
#pragma once

#include <QFile>
#include <QMutex>
#include <QString>
#include <QTextStream>

/**
 * @brief Singleton thread-safe logger.
 *
 * Write all application events to a UTF-8 text file with timestamp and level.
 * Safe to call from any thread (Scanner runs in QThread).
 *
 * Usage:
 *   Logger::instance()->open(path);
 *   Logger::instance()->info("Scan started");
 */
class Logger {
public:
    static Logger* instance();

    /** Open (append) the log file. Must be called once from the main thread. */
    bool open(const QString& filePath);

```



```

void info(const QString& message);
void warning(const QString& message);
void error(const QString& message);

/** Returns the path passed to open(), or empty if not opened. */
QString filePath() const { return m_logFile.fileName(); }

```

```

private:
    Logger() = default;
    ~Logger();

    void write(const QString& level, const QString& message);

    static Logger* s_instance;

    QFile      m_logFile;
    QTextStream m_stream;
    QMutex      m_mutex;
};
...

```

```

main.cpp:
...

```

```

#include <QApplication>
#include <QDir>
#include <QFile>
#include <QDebug>
#include <QProcessEnvironment>
#include <QTextStream>
#include <QThread>
#include <iostream>

```

```

#include "scanner.h"
#include "signaturedb.h"

```

```

#include "logger.h"
#include "mainwindow.h"

```

```

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);
    app.setApplicationName(QStringLiteral("AntivirusMVP"));
    app.setOrganizationName(QStringLiteral("NULP-KI307"));
    app.setApplicationVersion(QStringLiteral("1.0.0"));

    const QString appData = QProcessEnvironment::systemEnvironment().value(QStringLiteral("APPDATA"));
    const QString appDataDir = QDir(appData).filePath(QStringLiteral("AntivirusMVP"));
    const QString logPath = QDir(appDataDir).filePath(QStringLiteral("antivirus.log"));
    if (Logger::instance()->open(logPath)) {
        Logger::instance()->info(QStringLiteral("Application started"));
    }
}

```

```

#if !defined(NDEBUG)
    const QString tempScanDir = QDir(QDir::tempPath()).filePath(QStringLiteral("AntivirusMVP_US05"));
    QDir().mkpath(tempScanDir);

```

```

    const QString eicarPath = QDir(tempScanDir).filePath(QStringLiteral("eicar_test.txt"));
    const auto writeEicarFile = [&eicarPath]() -> bool {
        QFile eicarFile(eicarPath);
        if (eicarFile.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate)) {
            QTextStream out(&eicarFile);
            out << QStringLiteral("X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*");

```

```

        eicarFile.close();
        return true;
    } else {
        qWarning() << "US-05: cannot create EICAR file:" << eicarPath;
        Logger::instance()->warning(QStringLiteral("US-05: cannot create EICAR file: ") + eicarPath);
        return false;
    }
};
writeEicarFile();

const QStringList signatureCandidates = {
    QStringLiteral("signatures.db"),
    QDir(QCoreApplication::applicationDirPath()).filePath(QStringLiteral("signatures.db")),
    QDir(QCoreApplication::applicationDirPath()).filePath(QStringLiteral("../..\\signatures.db"))
};

QString signatureDbPath;
for (const QString& candidate : signatureCandidates) {
    if (QFile::exists(candidate)) {
        signatureDbPath = candidate;
        break;
    }
}

SignatureDB signatureDb;
if (!signatureDbPath.isEmpty() && signatureDb.load(signatureDbPath)) {
    Scanner scanner;
    scanner.setDirectory(tempScanDir);
    scanner.setSignatureDB(&signatureDb);

    bool threatDetected = false;

    QObject::connect(&scanner, &Scanner::logMessage,
        [](const QString& level, const QString& message) {
            if (level == QStringLiteral("INFO")) {
                Logger::instance()->info(message);
            } else if (level == QStringLiteral("WARNING")) {
                Logger::instance()->warning(message);
            } else {
                Logger::instance()->error(message);
            }
        });

    QObject::connect(&scanner, &Scanner::threatFound,
        [&threatDetected](const QString& filePath, const QString& threatName, const QString& md5)
    {
        threatDetected = true;
        qInfo().noquote() << QStringLiteral("[US-05] THREAT: %1 in %2 (MD5: %3)")
            .arg(threatName, filePath, md5);
        std::cout << "[US-05] THREAT: "
            << threatName.toStdString()
            << " in "
            << filePath.toStdString()
            << " (MD5: "
            << md5.toStdString()
            << ")" << std::endl;
    });

    QObject::connect(&scanner, &Scanner::finished,
        [](int totalFiles, int threatsFound) {
            qInfo().noquote() << QStringLiteral("[US-05] Scan finished: %1 files, %2 threats")
                .arg(totalFiles)
                .arg(threatsFound);
        });
}

```

```

        std::cout << "[US-05] Scan finished: "
            << totalFiles
            << " files, "
            << threatsFound
            << " threats" << std::endl;
    });

    for (int attempt = 0; attempt < 3 && !threatDetected; ++attempt) {
        if (!QFile::exists(eicarPath)) {
            writeEicarFile();
        }
        scanner.run();
        if (!threatDetected) {
            QThread::msleep(150);
        }
    }

    if (!threatDetected) {
        Logger::instance()->warning(QStringLiteral("US-05: EICAR file was not detected"));
        std::cout << "[US-05] EICAR file was not detected" << std::endl;
    }
    } else {
        qWarning() << "US-05: cannot load signatures.db";
        Logger::instance()->warning(QStringLiteral("US-05: cannot load signatures.db"));
        std::cout << "[US-05] Cannot load signatures.db" << std::endl;
    }
}
#endif

MainWindow w;
w.show();
const int rc = app.exec();
Logger::instance()->info(QStringLiteral("Application stopped"));
return rc;
}

...

```

mainwindow.cpp:

```

...
#include "mainwindow.h"

#include <QApplication>
#include <QDateTime>
#include <QDesktopServices>
#include <QDir>
#include <QFile>
#include <QFileDialog>
#include <QFileInfo>
#include <QLabel>
#include <QLineEdit>
#include <QMessageBox>
#include <QPlainTextEdit>
#include <QProcessEnvironment>
#include <QProgressBar>
#include <QPushButton>
#include <QStandardPaths>
#include <QTabWidget>
#include <QTableWidget>
#include <QTextStream>
#include <QThread>
#include <QUrl>
#include <QHeaderView>
#include <QHBoxLayout>

```

```

#include <QVBoxLayout>
#include <QWidget>

#include "logger.h"
#include "processlist.h"
#include "quarantine.h"
#include "reportwriter.h"
#include "scanner.h"
#include "signaturedb.h"

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent) {
    setWindowTitle(QStringLiteral("AntivirusMVP 1.0.0"));
    setMinimumSize(800, 600);
    resize(1000, 700);

    m_signatureDb = new SignatureDB();
    m_processList = new ProcessList();
    m_quarantine = new Quarantine(QDir(appDataDir()).filePath(QStringLiteral("Quarantine")));

    loadWhitelist();
    buildUi();
}

MainWindow::~MainWindow() {
    if (m_scanner) {
        m_scanner->stop();
    }
    if (m_scanThread) {
        m_scanThread->quit();
        m_scanThread->wait(3000);
    }
    delete m_signatureDb;
    delete m_processList;
    delete m_quarantine;
}

void MainWindow::buildUi() {
    m_tabs = new QTabWidget(this);
    m_tabs->addTab(buildScanTab(), QStringLiteral("Сканування"));
    m_tabs->addTab(buildProcessTab(), QStringLiteral("Процеси"));
    m_tabs->addTab(buildQuarantineTab(), QStringLiteral("Карантин"));
    m_tabs->addTab(buildLogTab(), QStringLiteral("Журнал"));
    m_tabs->addTab(buildSettingsTab(), QStringLiteral("Налаштування"));

    connect(m_tabs, &QTabWidget::currentChanged, this, [this](int index) {
        if (index == 2) {
            refreshQuarantine();
        } else if (index == 3) {
            refreshLogView();
        }
    });

    setCentralWidget(m_tabs);
}

QWidget* MainWindow::buildPlaceholderTab(const QString& title) const {
    auto* page = new QWidget;
    auto* layout = new QVBoxLayout(page);
    auto* label = new QLabel(QStringLiteral("Вкладка \"") + title + QStringLiteral("\" буде реалізована в наступних User Stories."));
    label->setWordWrap(true);
    layout->addStretch();
}

```

```

    layout->addWidget(label);
    layout->addStretch();
    return page;
}

QWidget* MainWindow::buildScanTab() {
    auto* page = new QWidget;
    auto* root = new QVBoxLayout(page);

    auto* dirRow = new QHBoxLayout;
    m_scanDirEdit = new QLineEdit;
    m_scanDirEdit->setPlaceholderText(QStringLiteral("Оберіть директорію для сканування..."));
    auto* btnSelect = new QPushButton(QStringLiteral("Обрати..."));
    connect(btnSelect, &QPushButton::clicked, this, &MainWindow::selectScanDirectory);
    dirRow->addWidget(m_scanDirEdit);
    dirRow->addWidget(btnSelect);
    root->addLayout(dirRow);

    auto* actions = new QHBoxLayout;
    m_btnStart = new QPushButton(QStringLiteral("Розпочати сканування"));
    m_btnStop = new QPushButton(QStringLiteral("Зупинити"));
    m_btnOpenReport = new QPushButton(QStringLiteral("Відкрити звіт"));
    m_btnStop->setEnabled(false);
    m_btnOpenReport->setEnabled(false);
    connect(m_btnStart, &QPushButton::clicked, this, &MainWindow::startScan);
    connect(m_btnStop, &QPushButton::clicked, this, &MainWindow::stopScan);
    connect(m_btnOpenReport, &QPushButton::clicked, this, &MainWindow::openReport);
    actions->addWidget(m_btnStart);
    actions->addWidget(m_btnStop);
    actions->addStretch();
    actions->addWidget(m_btnOpenReport);
    root->addLayout(actions);

    m_scanProgress = new QProgressBar;
    m_scanProgress->setRange(0, 100);
    m_scanProgressText = new QLabel(QStringLiteral("Готово"));
    root->addWidget(m_scanProgress);
    root->addWidget(m_scanProgressText);

    m_scanTable = new QTableWidgetItem(0, 3);
    m_scanTable->setHorizontalHeaderLabels({
        QStringLiteral("Файл"),
        QStringLiteral("Статус"),
        QStringLiteral("Загроз")
    });
    m_scanTable->horizontalHeader()->setStretchLastSection(true);
    m_scanTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    m_scanTable->setSelectionBehavior(QAbstractItemView::SelectRows);
    root->addWidget(m_scanTable);

    return page;
}

QWidget* MainWindow::buildLogTab() {
    auto* page = new QWidget;
    auto* root = new QVBoxLayout(page);

    auto* label = new QLabel(QStringLiteral("Вміст antivirus.log"));
    root->addWidget(label);

    m_logView = new QPlainTextEdit;
    m_logView->setReadOnly(true);
    m_logView->setPlaceholderText(QStringLiteral("Журнал ще не створено..."));

```

```

root->addWidget(m_logView, 1);

refreshLogView();
return page;
}

```

```

QWidget* MainWindow::buildSettingsTab() {
    auto* page = new QWidget;
    auto* root = new QVBoxLayout(page);

    root->addWidget(new QLabel(QStringLiteral("Шляхи для виключення (один на рядок):")));
    m_whitelistEdit = new QPlainTextEdit;
    m_whitelistEdit->setPlainText(m_whitelist.join('\n'));
    root->addWidget(m_whitelistEdit, 1);

    auto* btnSave = new QPushButton(QStringLiteral("Зберегти"));
    connect(btnSave, &QPushButton::clicked, this, [this]() {
        if (saveWhitelist()) {
            QMessageBox::information(this, QStringLiteral("Налаштування"), QStringLiteral("Whitelist
збережено."));
        } else {
            QMessageBox::warning(this, QStringLiteral("Налаштування"), QStringLiteral("Не вдалося зберегти
whitelist."));
        }
    });
    root->addWidget(btnSave, 0, Qt::AlignRight);

    return page;
}

```

```

QWidget* MainWindow::buildProcessTab() {
    auto* page = new QWidget;
    auto* root = new QVBoxLayout(page);

    auto* btnRefresh = new QPushButton(QStringLiteral("Оновити список"));
    connect(btnRefresh, &QPushButton::clicked, this, &MainWindow::refreshProcesses);
    root->addWidget(btnRefresh);

    m_processTable = new QTableWidgetItem(0, 3);
    m_processTable->setHorizontalHeaderLabels({
        QStringLiteral("PID"),
        QStringLiteral("Ім'я"),
        QStringLiteral("Шлях")
    });
    m_processTable->horizontalHeader()->setStretchLastSection(true);
    m_processTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
    m_processTable->setSelectionBehavior(QAbstractItemView::SelectRows);
    root->addWidget(m_processTable);

    refreshProcesses();
    return page;
}

```

```

QWidget* MainWindow::buildQuarantineTab() {
    auto* page = new QWidget;
    auto* root = new QVBoxLayout(page);

    m_quarantineTable = new QTableWidgetItem(0, 4);
    m_quarantineTable->setHorizontalHeaderLabels({
        QStringLiteral("Файл"),
        QStringLiteral("Загроза"),
        QStringLiteral("Оригінальний шлях"),
        QStringLiteral("Дата")
    });
    root->addWidget(m_quarantineTable);

    return page;
}

```

```

});
m_quarantineTable->horizontalHeader()->setStretchLastSection(true);
m_quarantineTable->setEditTriggers(QAbstractItemView::NoEditTriggers);
m_quarantineTable->setSelectionBehavior(QAbstractItemView::SelectRows);
root->addWidget(m_quarantineTable);

auto* actions = new QHBoxLayout;
auto* btnRestore = new QPushButton(QStringLiteral("Відновити"));
auto* btnDelete = new QPushButton(QStringLiteral("Видалити назавжди"));
connect(btnRestore, &QPushButton::clicked, this, &MainWindow::quarantineRestore);
connect(btnDelete, &QPushButton::clicked, this, &MainWindow::quarantineDelete);
actions->addStretch();
actions->addWidget(btnRestore);
actions->addWidget(btnDelete);
root->addLayout(actions);

refreshQuarantine();
return page;
}

void MainWindow::selectScanDirectory() {
    const QString dir = QFileDialog::getExistingDirectory(
        this,
        QStringLiteral("Оберіть директорію"),
        QDir::homePath());
    if (!dir.isEmpty()) {
        m_scanDirEdit->setText(dir);
    }
}

bool MainWindow::loadWhitelist() {
    const QString whitelistPath = QDir(appDataDir()).filePath(QStringLiteral("whitelist.txt"));
    QFile file(whitelistPath);
    m_whitelist.clear();

    if (!file.exists()) {
        return true;
    }

    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        return false;
    }

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (!line.isEmpty()) {
            m_whitelist << line;
        }
    }
    file.close();

    if (m_whitelistEdit) {
        m_whitelistEdit->setPlainText(m_whitelist.join("\n"));
    }
    return true;
}

bool MainWindow::saveWhitelist() {
    const QString whitelistPath = QDir(appDataDir()).filePath(QStringLiteral("whitelist.txt"));
    QDir().mkpath(appDataDir());

    QFile file(whitelistPath);

```

```

    if (!file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate)) {
        return false;
    }

    QTextStream out(&file);
    const QStringList items = m_whitelistEdit ? m_whitelistEdit->toPlainText().split("\n", Qt::SkipEmptyParts) :
QStringList();
    m_whitelist = items;
    for (const QString& path : items) {
        out << path.trimmed() << '\n';
    }
    file.close();
    return true;
}

void MainWindow::refreshLogView() {
    if (!m_logView) {
        return;
    }

    const QString logPath = QDir(appDataDir()).filePath(QStringLiteral("antivirus.log"));
    QFile file(logPath);
    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        m_logView->setPlainText(QStringLiteral("Журнал ще не створено."));
        return;
    }

    QTextStream in(&file);
    m_logView->setPlainText(in.readAll());
}

bool MainWindow::loadSignatureDb() {
    if (m_signatureDb->count() > 0) {
        return true;
    }

    const QString appDir = QApplication::applicationDirPath();
    const QStringList candidates = {
        QStringLiteral("signatures.db"),
        QDir(appDir).filePath(QStringLiteral("signatures.db")),
        QDir(appDir).filePath(QStringLiteral("../..\\signatures.db"))
    };

    for (const QString& path : candidates) {
        if (QFile::exists(path) && m_signatureDb->load(path)) {
            Logger::instance()->info(QStringLiteral("SignatureDB loaded from: ") + path);
            return true;
        }
    }

    QMessageBox::warning(this,
        QStringLiteral("Сканування"),
        QStringLiteral("Не вдалося завантажити signatures.db"));
    return false;
}

void MainWindow::startScan() {
    const QString dir = m_scanDirEdit->text().trimmed();
    if (dir.isEmpty()) {
        QMessageBox::information(this, QStringLiteral("Сканування"), QStringLiteral("Оберіть директорію
для сканування."));
        return;
    }
}

```



```

if (!QDir(dir).exists()) {
    QMessageBox::warning(this, QStringLiteral("Сканування"), QStringLiteral("Директорія не існує."));
    return;
}
if (!loadSignatureDb()) {
    return;
}

m_detectedThreats.clear();
m_lastReportPath.clear();
m_scanTable->setRowCount(0);
m_scanProgress->setValue(0);
m_scanProgressText->setText(QStringLiteral("Сканування..."));
m_btnStart->setEnabled(false);
m_btnStop->setEnabled(true);
m_btnOpenReport->setEnabled(false);

m_scanThread = new QThread(this);
m_scanner = new Scanner();
m_scanner->setDirectory(dir);
m_scanner->setWhitelist(m_whitelist);
m_scanner->setSignatureDB(m_signatureDb);
m_scanner->moveToThread(m_scanThread);

connect(m_scanThread, &QThread::started, m_scanner, &Scanner::run);
connect(m_scanner, &Scanner::progress, this, &MainWindow::onScanProgress);
connect(m_scanner, &Scanner::threatFound, this, &MainWindow::onThreatFound);
connect(m_scanner, &Scanner::finished, this, &MainWindow::onScanFinished);
connect(m_scanner, &Scanner::finished, m_scanThread, &QThread::quit);

connect(m_scanner, &Scanner::logMessage, this, [](const QString& level, const QString& message) {
    if (level == QStringLiteral("INFO")) {
        Logger::instance()->info(message);
    } else if (level == QStringLiteral("WARNING")) {
        Logger::instance()->warning(message);
    } else {
        Logger::instance()->error(message);
    }
});

connect(m_scanThread, &QThread::finished, m_scanner, &QObject::deleteLater);
connect(m_scanThread, &QThread::finished, m_scanThread, &QObject::deleteLater);
connect(m_scanThread, &QThread::finished, this, [this]() {
    m_scanner = nullptr;
    m_scanThread = nullptr;
});

m_scanThread->start();
}

void MainWindow::stopScan() {
    if (m_scanner) {
        m_scanner->stop();
    }
}

void MainWindow::onScanProgress(int current, int total) {
    if (total > 0) {
        m_scanProgress->setValue((current * 100) / total);
    }
    m_scanProgressText->setText(QStringLiteral("Перевірено %1 / %2 файлів").arg(current).arg(total));
}

```

```

void MainWindow::onThreatFound(const QString& filePath,
                               const QString& threatName,
                               const QString& md5Hex) {
    const int row = m_scanTable->rowCount();
    m_scanTable->insertRow(row);

    auto* fileItem = new QTableWidgetItem(filePath);
    auto* statusItem = new QTableWidgetItem(QStringLiteral("Загроза"));
    auto* threatItem = new QTableWidgetItem(threatName);

    fileItem->setForeground(Qt::red);
    statusItem->setForeground(Qt::red);
    threatItem->setForeground(Qt::red);

    m_scanTable->setItem(row, 0, fileItem);
    m_scanTable->setItem(row, 1, statusItem);
    m_scanTable->setItem(row, 2, threatItem);

    m_detectedThreats.append({filePath, threatName, md5Hex});

    if (!m_quarantine) {
        return;
    }

    const auto answer = QMessageBox::question(
        this,
        QStringLiteral("Загроза виявлена"),
        QStringLiteral("Виявлено загрозу:\n%1\n\nФайл:\n%2\n\nІзолювати файл у карантин?"),
        .arg(threatName, filePath),
        QMessageBox::Yes | QMessageBox::No,
        QMessageBox::Yes);

    if (answer != QMessageBox::Yes) {
        return;
    }

    if (!m_quarantine->isolate(filePath, threatName)) {
        QMessageBox::warning(this,
                               QStringLiteral("Карантин"),
                               QStringLiteral("Не вдалося ізолювати файл у карантин."));
        Logger::instance()->warning(QStringLiteral("Failed to quarantine: ") + filePath);
        return;
    }

    Logger::instance()->info(QStringLiteral("Quarantined: ") + filePath);
    refreshQuarantine();
}

bool MainWindow::writeScanReport(int totalFiles, int threatsFound) {
    const QString reportsDir = QDir(appDataDir()).filePath(QStringLiteral("Reports"));
    QDir().mkpath(reportsDir);

    const QString fileName = QStringLiteral("scan_")
        + QDateTime::currentDateTime().toString(QStringLiteral("yyyyMMdd_HH:mm:ss"))
        + QStringLiteral(".txt");
    m_lastReportPath = QDir(reportsDir).filePath(fileName);

    ScanResult result;
    result.directory = m_scanDirEdit->text().trimmed();
    result.totalFiles = totalFiles;
    result.threatsFound = threatsFound;
    result.threats = m_detectedThreats;
}

```

```

    if (!ReportWriter::write(m_lastReportPath, result)) {
        m_lastReportPath.clear();
        return false;
    }
    return true;
}

QString MainWindow::appDataDir() const {
    const QString appData =
        QProcessEnvironment::systemEnvironment().value(QStringLiteral("APPDATA"));
    if (!appData.isEmpty()) {
        return QDir(appData).filePath(QStringLiteral("AntivirusMVP"));
    }
    return QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
}

void MainWindow::onScanFinished(int totalFiles, int threatsFound) {
    m_scanProgress->setValue(100);
    m_scanProgressText->setText(
        QStringLiteral("Завершено: %1 файлів, %2 загроз").arg(totalFiles).arg(threatsFound));

    m_btnStart->setEnabled(true);
    m_btnStop->setEnabled(false);

    if (writeScanReport(totalFiles, threatsFound)) {
        m_btnOpenReport->setEnabled(true);
    }
}

void MainWindow::openReport() {
    if (m_lastReportPath.isEmpty() || !QFile::exists(m_lastReportPath)) {
        QMessageBox::information(this, QStringLiteral("Звіт"), QStringLiteral("Звіт ще не створено."));
        return;
    }
    QDesktopServices::openUrl(QUrl::fromLocalFile(m_lastReportPath));
}

void MainWindow::refreshProcesses() {
    if (!m_processList || !m_processTable) {
        return;
    }

    const auto processes = m_processList->snapshot();
    m_processTable->setRowCount(0);

    for (const ProcessInfo& process : processes) {
        const int row = m_processTable->rowCount();
        m_processTable->insertRow(row);

        auto* pidItem = new QTableWidgetItem(QString::number(process.pid));
        auto* nameItem = new QTableWidgetItem(process.name);
        auto* pathItem = new QTableWidgetItem(process.executablePath);

        if (process.isSuspicious) {
            pidItem->setForeground(Qt::red);
            nameItem->setForeground(Qt::red);
            pathItem->setForeground(Qt::red);
        }

        m_processTable->setItem(row, 0, pidItem);
        m_processTable->setItem(row, 1, nameItem);
        m_processTable->setItem(row, 2, pathItem);
    }
}

```

```

}

void MainWindow::refreshQuarantine() {
    if (!m_quarantine || !m_quarantineTable) {
        return;
    }

    const auto items = m_quarantine->entries();
    m_quarantineTable->setRowCount(0);

    for (const QuarantineEntry& entry : items) {
        const int row = m_quarantineTable->rowCount();
        m_quarantineTable->insertRow(row);
        m_quarantineTable->setItem(row, 0, new QTableWidgetItem(entry.fileName));
        m_quarantineTable->setItem(row, 1, new QTableWidgetItem(entry.threatName));
        m_quarantineTable->setItem(row, 2, new QTableWidgetItem(entry.originalPath));
        m_quarantineTable->setItem(row, 3, new QTableWidgetItem(entry.date));
    }
}

void MainWindow::quarantineRestore() {
    if (!m_quarantineTable || !m_quarantine) {
        return;
    }
    const int row = m_quarantineTable->currentRow();
    if (row < 0) {
        QMessageBox::information(this, QStringLiteral("Карантин"), QStringLiteral("Оберіть запис для
відновлення."));
        return;
    }

    if (!m_quarantine->restore(row)) {
        QMessageBox::warning(this, QStringLiteral("Карантин"), QStringLiteral("Не вдалося відновити
файл."));
    }
    refreshQuarantine();
}

void MainWindow::quarantineDelete() {
    if (!m_quarantineTable || !m_quarantine) {
        return;
    }
    const int row = m_quarantineTable->currentRow();
    if (row < 0) {
        QMessageBox::information(this, QStringLiteral("Карантин"), QStringLiteral("Оберіть запис для
видалення."));
        return;
    }

    if (QMessageBox::question(this,
        QStringLiteral("Карантин"),
        QStringLiteral("Видалити файл назавжди?"),
        QMessageBox::Yes | QMessageBox::No)
        != QMessageBox::Yes) {
        return;
    }

    if (!m_quarantine->removeEntry(row)) {
        QMessageBox::warning(this, QStringLiteral("Карантин"), QStringLiteral("Не вдалося видалити
файл."));
    }
    refreshQuarantine();
}

```

```
...
```

```
mainwindow.h:
```

```
...
```

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
```

```
#include <QList>
#include <QMainWindow>
#include <QStringList>
```

```
#include "reportwriter.h"
```

```
class QTabWidget;
class QLabel;
class QLineEdit;
class QPushButton;
class QPlainTextEdit;
class QProgressBar;
class QTableWidget;
class QThread;
class QWidget;
```

```
class Scanner;
class SignatureDB;
class ProcessList;
class Quarantine;
```

```
class MainWindow : public QMainWindow {
    Q_OBJECT
```

```
public:
```

```
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
```

```
private slots:
```

```
    void selectScanDirectory();
    void startScan();
    void stopScan();
    void openReport();
    void refreshProcesses();
    void refreshQuarantine();
    void quarantineRestore();
    void quarantineDelete();

    void onScanProgress(int current, int total);
    void onThreatFound(const QString& filePath,
                      const QString& threatName,
                      const QString& md5Hex);
    void onScanFinished(int totalFiles, int threatsFound);
```

```
private:
```

```
    void buildUi();
    QWidget* buildScanTab();
    QWidget* buildProcessTab();
    QWidget* buildQuarantineTab();
    QWidget* buildLogTab();
    QWidget* buildSettingsTab();
    QWidget* buildPlaceholderTab(const QString& title) const;
```

```
    bool loadSignatureDb();
    bool loadWhitelist();
```

```

bool saveWhitelist();
void refreshLogView();
bool writeScanReport(int totalFiles, int threatsFound);
QString appDataDir() const;

```

```
QTabWidget* m_tabs = nullptr;
```

```

QLineEdit* m_scanDirEdit = nullptr;
QPushButton* m_btnStart = nullptr;
QPushButton* m_btnStop = nullptr;
QPushButton* m_btnOpenReport = nullptr;
QProgressBar* m_scanProgress = nullptr;
QLabel* m_scanProgressText = nullptr;
QTableWidget* m_scanTable = nullptr;
QTableWidget* m_processTable = nullptr;
QTableWidget* m_quarantineTable = nullptr;
QPlainTextEdit* m_logView = nullptr;
QPlainTextEdit* m_whitelistEdit = nullptr;

```

```

QThread* m_scanThread = nullptr;
Scanner* m_scanner = nullptr;
SignatureDB* m_signatureDb = nullptr;
ProcessList* m_processList = nullptr;
Quarantine* m_quarantine = nullptr;

```

```

QString m_lastReportPath;
QList<ThreatEntry> m_detectedThreats;
QStringList m_whitelist;

```

```
};
```

```
#endif // MAINWINDOW_H
```

```
...
```

```
processlist.cpp:
```

```
...
```

```
#include "processlist.h"
```

```
#define WIN32_LEAN_AND_MEAN
```

```
#include <windows.h>
```

```
#include <tlhelp32.h>
```

```

QVector<ProcessInfo> ProcessList::snapshot() {
    QVector<ProcessInfo> result;

```

```

    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE)
        return result;

```

```

    PROCESSENTRY32W pe = {};
    pe.dwSize = sizeof(pe);

```

```

    if (Process32FirstW(hSnap, &pe)) {
        do {
            ProcessInfo info;
            info.pid = static_cast<uint32_t>(pe.th32ProcessID);
            info.name = QString::fromWCharArray(pe.szExeFile);

```

```

            HANDLE hProc = OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, FALSE,
pe.th32ProcessID);
            if (hProc) {
                wchar_t buf[MAX_PATH] = {};
                DWORD sz = MAX_PATH;

```

```

        if (QueryFullProcessImageNameW(hProc, 0, buf, &sz))
            info.executablePath = QString::fromWCharArray(buf);
        CloseHandle(hProc);
    }

    if (info.executablePath.isEmpty())
        info.executablePath = QStringLiteral("N/A");

    info.isSuspicious = isSuspiciousPath(info.executablePath);
    result.append(std::move(info));

} while (Process32NextW(hSnap, &pe));
}

CloseHandle(hSnap);
return result;
}

bool ProcessList::isSuspiciousPath(const QString& path) {
    if (path == QLatin1String("N/A"))
        return false;

    const QString lower = path.toLower();
    return lower.contains(QLatin1String("\\temp\\"))
        || lower.contains(QLatin1String("\\tmp\\"))
        || lower.contains(QLatin1String("\\appdata\\local\\temp\\"))
        || lower.contains(QLatin1String("\\downloads\\"));
}

...

processlist.h:
...
#pragma once

#include <QVector>
#include <QString>
#include <cstdint>

struct ProcessInfo {
    uint32_t pid = 0;
    QString name;
    QString executablePath;
    bool isSuspicious = false;
};

/**
 * @brief Windows process list via CreateToolhelp32Snapshot.
 *
 * Uses PROCESS_QUERY_LIMITED_INFORMATION so that even restricted processes
 * can be enumerated. Paths unavailable due to access restrictions are
 * reported as "N/A" rather than causing a crash.
 */
class ProcessList {
public:
    QVector<ProcessInfo> snapshot();

private:
    static bool isSuspiciousPath(const QString& path);
};

...

```

```

quarantine.cpp:
```
#include "quarantine.h"

#include <QDateTime>
#include <QDir>
#include <QFile>
#include <QFileInfo>
#include <QJsonDocument>
#include <QJsonObject>
#include <QStorageInfo>

Quarantine::Quarantine(const QString& quarantineDir) : m_dir(quarantineDir) {
 QDir().mkpath(m_dir);
 loadIndex();
}

bool Quarantine::isolate(const QString& filePath, const QString& threatName) {
 QStorageInfo si(m_dir);
 if (si.bytesFree() < 100LL * 1024 * 1024)
 return false;

 const QFileInfo fi(filePath);
 const QString dest = generateDest(fi.fileName());

 if (!QFile::rename(filePath, dest))
 return false;

 QJsonObject obj;
 obj[QStringLiteral("originalPath")] = filePath;
 obj[QStringLiteral("quarantinedPath")] = dest;
 obj[QStringLiteral("threatName")] = threatName;
 obj[QStringLiteral("fileName")] = fi.fileName();
 obj[QStringLiteral("date")] =
 QDateTime::currentDateTime().toString(QStringLiteral("dd.MM.yyyy HH:mm:ss"));

 m_index.append(obj);
 saveIndex();
 return true;
}

bool Quarantine::restore(int index) {
 if (index < 0 || index >= m_index.size())
 return false;

 const QJsonObject obj = m_index.at(index).toObject();
 const QString src = obj[QStringLiteral("quarantinedPath")].toString();
 const QString dest = obj[QStringLiteral("originalPath")].toString();

 QDir().mkpath(QFileInfo(dest).absolutePath());
 if (!QFile::rename(src, dest))
 return false;

 m_index.removeAt(index);
 saveIndex();
 return true;
}

bool Quarantine::removeEntry(int index) {
 if (index < 0 || index >= m_index.size())
 return false;

 QFile::remove(

```



```

 m_index.at(index).toObject()[QStringLiteral("quarantinedPath")].toString());
 m_index.removeAt(index);
 saveIndex();
 return true;
}

```

```

QVector<QuarantineEntry> Quarantine::entries() const {
 QVector<QuarantineEntry> result;
 result.reserve(m_index.size());

 for (const QJsonValue& v : m_index) {
 const QJsonObject obj = v.toObject();
 QuarantineEntry e;
 e.originalPath = obj[QStringLiteral("originalPath")].toString();
 e.quarantinedPath = obj[QStringLiteral("quarantinedPath")].toString();
 e.threatName = obj[QStringLiteral("threatName")].toString();
 e.fileName = obj[QStringLiteral("fileName")].toString();
 e.date = obj[QStringLiteral("date")].toString();
 result.append(std::move(e));
 }
 return result;
}

```

```

void Quarantine::loadIndex() {
 QFile f(m_dir + QLatin1String("/quarantine_index.json"));
 if (!f.open(QIODevice::ReadOnly))
 return;

```

```

 m_index = QJsonDocument::fromJson(f.readAll()).array();
}

```

```

void Quarantine::saveIndex() {
 QFile f(m_dir + QLatin1String("/quarantine_index.json"));
 if (!f.open(QIODevice::WriteOnly | QIODevice::Truncate))
 return;

```

```

 f.write(QJsonDocument(m_index).toJson());
}

```

```

QString Quarantine::generateDest(const QString& fileName) const {
 const QFileInfo fi(fileName);
 const QString ts = QDateTime::currentDateTime()
 .toString(QStringLiteral("yyyyMMdd_HH:mm:ss"));
 const QString base = fi.completeBaseName().isEmpty() ? fileName
 : fi.completeBaseName();
 const QString ext = fi.suffix().isEmpty()
 ? QString()
 : QLatin1Char('.') + fi.suffix();

 return m_dir + QLatin1Char('/') + base + QLatin1Char('_') + ts + ext;
}

```

```

...

```

```

quarantine.h:
...

```

```

#pragma once

```

```

#include <QJsonArray>
#include <QString>
#include <QVector>

```

```

struct QuarantineEntry {

```

```

 QString originalPath;
 QString quarantinedPath;
 QString threatName;
 QString fileName;
 QString date;
};

class Quarantine {
public:
 explicit Quarantine(const QString& quarantineDir);

 bool isolate(const QString& filePath, const QString& threatName);
 bool restore(int index);
 bool removeEntry(int index);

 QVector<QuarantineEntry> entries() const;

private:
 void loadIndex();
 void saveIndex();
 QString generateDest(const QString& fileName) const;

 QString m_dir;
 QJsonArray m_index;
};

```

```
...
```

```
reportwriter.cpp:
```

```
...
```

```
#include "reportwriter.h"
```

```
#include <QDateTime>
```

```
#include <QFile>
```

```
#include <QTextStream>
```

```

bool ReportWriter::write(const QString& path, const ScanResult& result) {
 QFile outFile(path);
 if (!outFile.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate)) {
 return false;
 }

 QTextStream out(&outFile);
 out << "AntivirusMVP Scan Report\n";
 out << "Time: " << QDateTime::currentDateTime().toString(Qt::ISODate) << "\n";
 out << "Directory: " << result.directory << "\n";
 out << "Total files: " << result.totalFiles << "\n";
 out << "Threats found: " << result.threatsFound << "\n\n";
 out << "Threat list:\n";

 if (result.threats.isEmpty()) {
 out << "(none)\n";
 } else {
 for (const ThreatEntry& threat : result.threats) {
 out << "- " << threat.filePath << "\n";
 out << " Threat: " << threat.threatName << "\n";
 out << " MD5: " << threat.md5Hex << "\n";
 }
 }

 out.flush();
 return out.status() == QTextStream::Ok;
}

```

```
...
```

```
reportwriter.h:
```

```
...
```

```
#ifndef REPORTWRITER_H
#define REPORTWRITER_H
```

```
#include <QList>
#include <QString>
```

```
struct ThreatEntry {
 QString filePath;
 QString threatName;
 QString md5Hex;
};
```

```
struct ScanResult {
 QString directory;
 int totalFiles = 0;
 int threatsFound = 0;
 QList<ThreatEntry> threats;
};
```

```
class ReportWriter {
public:
 static bool write(const QString& path, const ScanResult& result);
};
```

```
#endif // REPORTWRITER_H
```

```
...
```

```
scanner.cpp:
```

```
...
```

```
#include "scanner.h"
```

```
#include <QCryptographicHash>
#include <QDirIterator>
#include <QFile>
```

```
Scanner::Scanner(QObject* parent) : QObject(parent) {}
```

```
// -- Private helper -----
```

```
QString Scanner::computeMd5(const QString& filePath) {
 QFile file(filePath);
 if (!file.open(QIODevice::ReadOnly))
 return {};

 QCryptographicHash hasher(QCryptographicHash::Md5);
 hasher.addData(&file);
 return QString::fromLatin1(hasher.result().toHex());
}
```

```
// -- Main scan loop -----
```

```
void Scanner::run() {
 m_running.store(true);

 int total = 0;
 {
 QDirIterator counter(m_directory,
```

```

 QDir::Files | QDir::NoSymLinks,
 QDirIterator::Subdirectories);
while (counter.hasNext()) {
 counter.next();
 ++total;
}
}

int current = 0;
int threats = 0;

QDirIterator it(m_directory,
 QDir::Files | QDir::NoSymLinks,
 QDirIterator::Subdirectories);

while (it.hasNext() && m_running.load()) {
 const QString filePath = it.next();
 emit progress(++current, total);

 bool skip = false;
 for (const QString& whitelistPath : std::as_const(m_whitelist)) {
 if (filePath.startsWith(whitelistPath, Qt::CaseInsensitive)) {
 skip = true;
 break;
 }
 }
 if (skip) {
 continue;
 }

 const QString md5 = computeMd5(filePath);
 if (md5.isEmpty()) {
 emit logMessage(QStringLiteral("WARNING"),
 QStringLiteral("Cannot open: ") + filePath);
 continue;
 }

 if (m_db) {
 const QString threat = m_db->lookup(md5);
 if (!threat.isEmpty()) {
 ++threats;
 emit threatFound(filePath, threat, md5);
 emit logMessage(QStringLiteral("WARNING"),
 QStringLiteral("THREAT [%1] in: %2")
 .arg(threat, filePath));
 }
 }
}

emit finished(current, threats);
emit logMessage(QStringLiteral("INFO"),
 QStringLiteral("Scan done: %1 files, %2 threats")
 .arg(current)
 .arg(threats));
}

...

scanner.h:
...

#pragma once

#include <QObject>

```

```

#include <QStringList>
#include <atomic>

#include "signaturedb.h"

/**
 * @brief Recursive file-system scanner.
 *
 * Designed to run inside a QThread:
 * scanner->moveToThread(thread);
 * connect(thread, &QThread::started, scanner, &Scanner::run);
 *
 * Emits progress(), threatFound(), and finished() to be connected
 * to GUI slots via Qt::QueuedConnection (automatic for cross-thread signals).
 */
class Scanner : public QObject {
 Q_OBJECT

public:
 explicit Scanner(QObject* parent = nullptr);

 void setDirectory(const QString& path) { m_directory = path; }
 void setWhitelist(const QStringList& paths) { m_whitelist = paths; }
 void setSignatureDB(SignatureDB* db) { m_db = db; }

 /** Thread-safe stop: may be called from the GUI thread. */
 void stop() { m_running.store(false); }

public slots:
 void run();

signals:
 /** Fired for every file processed. */
 void progress(int current, int total);

 /** Fired when a file matches a signature. */
 void threatFound(const QString& filePath,
 const QString& threatName,
 const QString& md5Hex);

 /** Fired once when scan completes or is stopped. */
 void finished(int totalFiles, int threatsFound);

 /** General log messages (routed to Logger in GUI thread). */
 void logMessage(const QString& level, const QString& message);

private:
 static QString computeMd5(const QString& filePath);

 QString m_directory;
 QStringList m_whitelist;
 SignatureDB* m_db = nullptr;
 std::atomic<bool> m_running{false};
};

...

signaturedb.cpp:
...
#include "signaturedb.h"

#include <QFile>
#include <QRegularExpression>

```

```

#include <QTextStream>

bool SignatureDB::load(const QString& filePath) {
 QFile file(filePath);
 if (!file.open(QIODevice::ReadOnly | QIODevice::Text))
 return false;

 m_signatures.clear();

 static const QRegularExpression ws(QRegularExpressionLiteral(R"(\s+)"));
 QTextStream in(&file);
 while (!in.atEnd()) {
 const QString line = in.readLine().trimmed();
 if (line.isEmpty() || line.startsWith(QLatin1Char('#')))
 continue;

 const QStringList parts = line.split(ws, Qt::SkipEmptyParts);
 if (parts.size() >= 2)
 m_signatures.insert(parts.at(0).toLower(),
 parts.mid(1).join(QLatin1Char(' ')));
 }
 return !m_signatures.isEmpty();
}

QString SignatureDB::lookup(const QString& md5Hex) const {
 return m_signatures.value(md5Hex.toLower());
}

...

signedb.h:
...

#pragma once

#include <QHash>
#include <QString>

/**
 * @brief Loads and queries the MD5 signature database.
 *
 * File format (signatures.db):
 * # comment line
 * 44d88612fea8a8f36de82e1278abb02f EICAR-Standard-Test-File
 *
 * The first whitespace-delimited token is the lowercase MD5 hex.
 * The rest of the line is the threat name.
 */
class SignatureDB {
public:
 /**
 * @brief Load signatures from file.
 * @return true if at least one signature was loaded.
 */
 bool load(const QString& filePath);

 /**
 * @brief Look up a threat name by MD5 hex string.
 * @return Threat name, or empty QString if not found.
 */
 QString lookup(const QString& md5Hex) const;

 int count() const { return static_cast<int>(m_signatures.size()); }

```

```

private:
 QHash<QString, QString> m_signatures; // lowercase MD5 -> threat name
};

...

test_quarantine.cpp:
...
#include <QtTest/QtTest>

#include <QDir>
#include <QFile>
#include <QTemporaryDir>
#include <QTextStream>

#include "quarantine.h"

class TestQuarantine : public QObject {
 Q_OBJECT

private slots:
 void isolateAndRestore();
};

void TestQuarantine::isolateAndRestore() {
 QTemporaryDir tempDir;
 QVERIFY(tempDir.isValid());

 const QString quarantineDir = QDir(tempDir.path()).filePath(QStringLiteral("quarantine"));
 const QString filePath = QDir(tempDir.path()).filePath(QStringLiteral("sample.txt"));
 QFile file(filePath);
 QVERIFY(file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate));
 QTextStream out(&file);
 out << QStringLiteral("sample");
 file.close();

 Quarantine quarantine(quarantineDir);
 QVERIFY(quarantine.isolate(filePath, QStringLiteral("TestThreat")));
 QVERIFY(!QFile::exists(filePath));
 QCOMPARE(quarantine.entries().size(), 1);

 QVERIFY(quarantine.restore(0));
 QVERIFY(QFile::exists(filePath));
}

QTEST_MAIN(TestQuarantine)
#include "test_quarantine.moc"

...

test_scanner.cpp:
...
#include <QtTest/QtTest>

#include <QCoreApplication>
#include <QDir>
#include <QFile>
#include <QTemporaryDir>
#include <QTextStream>

#include "scanner.h"
#include "signaturedb.h"

```

```

class TestScanner : public QObject {
 Q_OBJECT

private slots:
 void detectsThreatInDirectory();
};

namespace {
QString findSignatureDbPath() {
 const QStringList candidates = {
 QStringLiteral("signatures.db"),
 QDir(QCoreApplication::applicationDirPath()).filePath(QStringLiteral("signatures.db")),
 QDir(QCoreApplication::applicationDirPath()).filePath(QStringLiteral("../..\\signatures.db"))
 };
 for (const QString& candidate : candidates) {
 if (QFile::exists(candidate))
 return candidate;
 }
 return {};
}
}

void TestScanner::detectsThreatInDirectory() {
 const QString signaturePath = findSignatureDbPath();
 QVERIFY2(!signaturePath.isEmpty(), "signatures.db was not found for tests");

 QTemporaryDir tempDir;
 QVERIFY(tempDir.isValid());
 const QString filePath = QDir(tempDir.path()).filePath(QStringLiteral("eicar_test.txt"));
 QFile file(filePath);
 QVERIFY(file.open(QIODevice::WriteOnly | QIODevice::Text | QIODevice::Truncate));
 QTextStream out(&file);
 out << QStringLiteral("X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-
FILE!$H+H*");
 file.close();

 SignatureDB db;
 QVERIFY(db.load(signaturePath));

 Scanner scanner;
 scanner.setDirectory(tempDir.path());
 scanner.setSignatureDB(&db);

 bool threatDetected = false;
 QObject::connect(&scanner, &Scanner::threatFound,
 [&threatDetected](const QString&, const QString&, const QString&) {
 threatDetected = true;
 });

 scanner.run();
 QVERIFY(threatDetected);
}

QTEST_MAIN(TestScanner)
#include "test_scanner.moc"

...

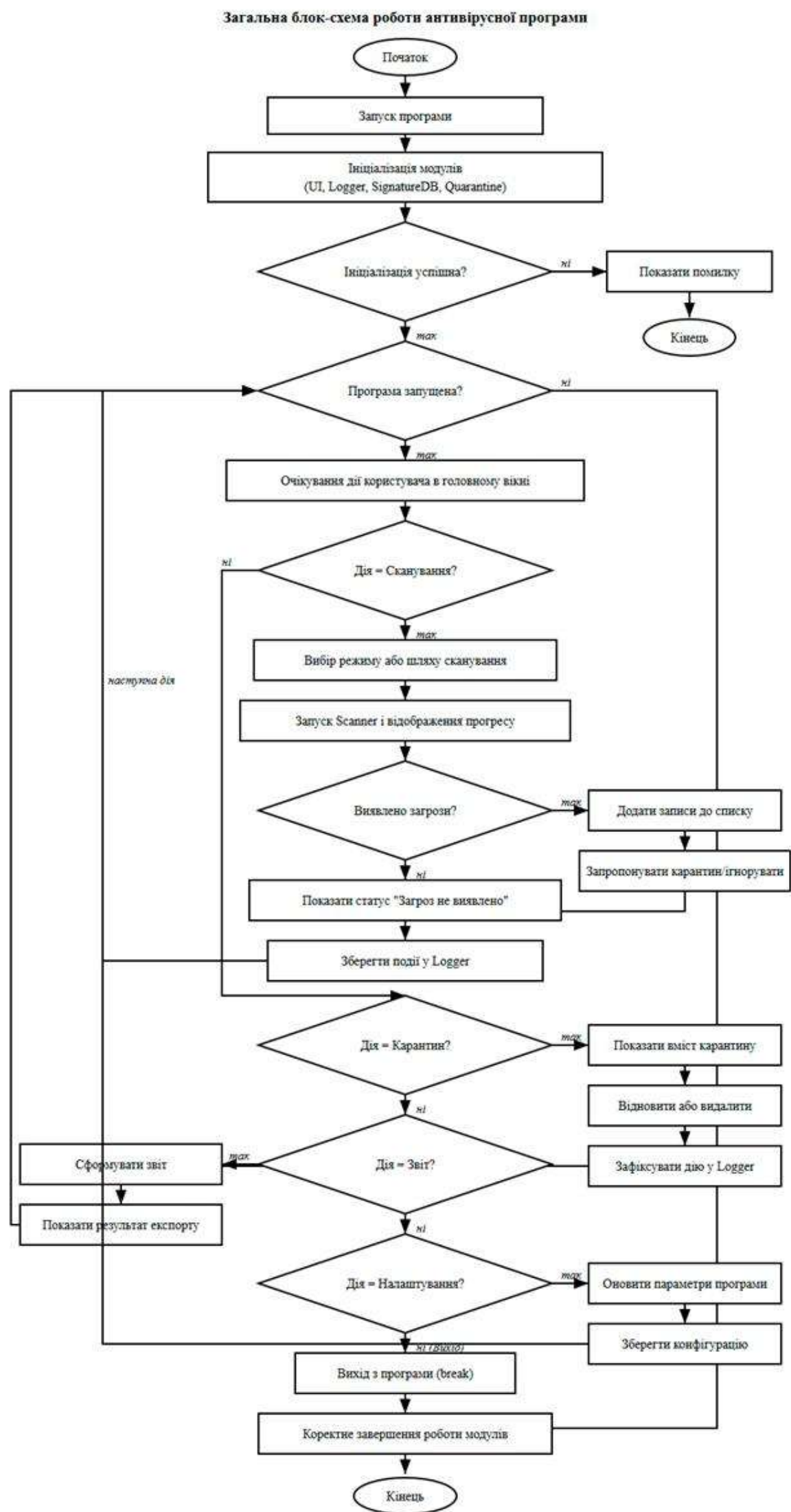
test_signatredb.cpp:
...
#include <QtTest/QtTest>

#include <QCoreApplication>

```







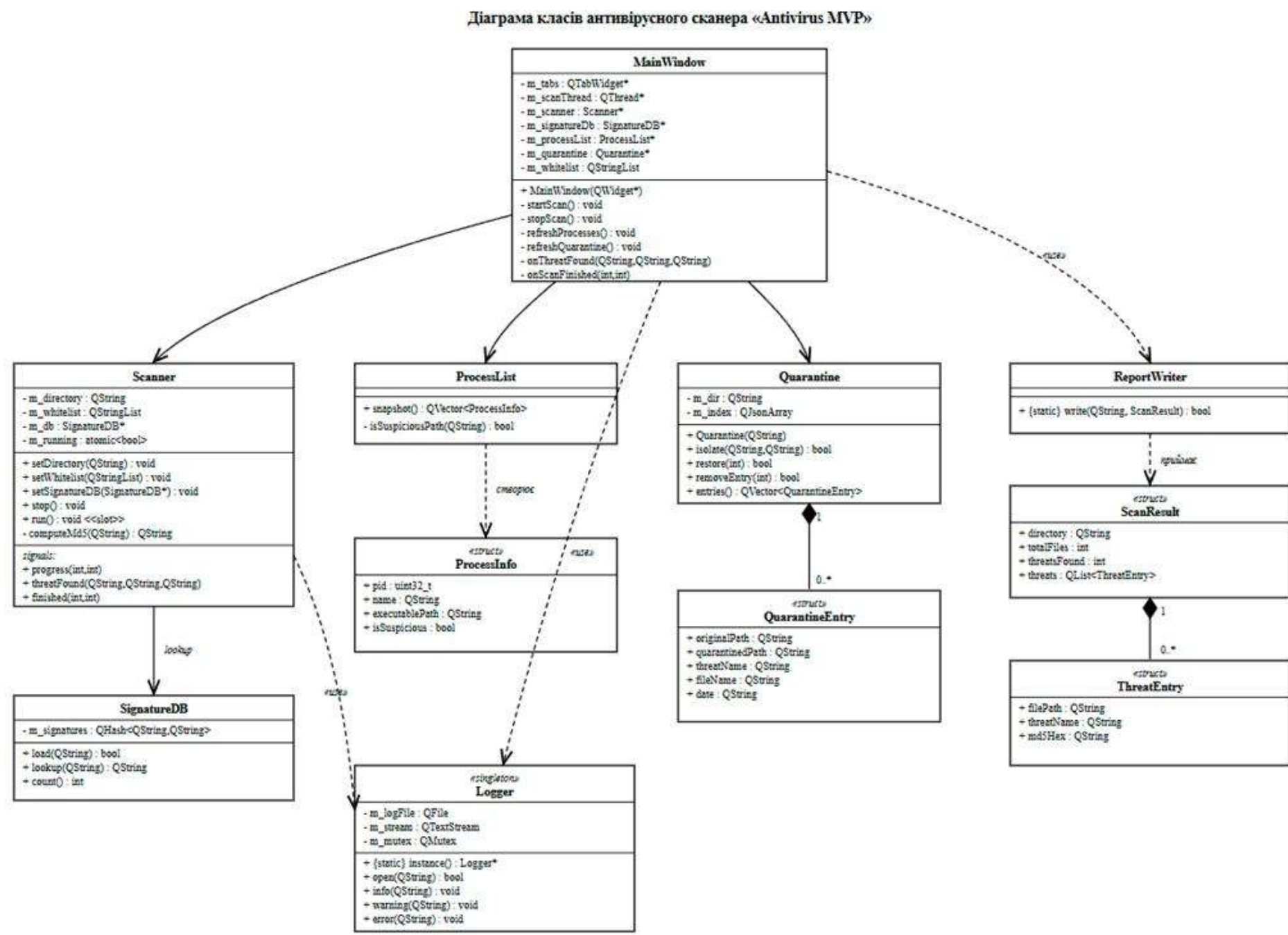
|                                     |               |          |        |      |                                                                                |                                       |   |           |         |
|-------------------------------------|---------------|----------|--------|------|--------------------------------------------------------------------------------|---------------------------------------|---|-----------|---------|
| Міністерство освіти і науки України |               |          |        |      | КУРСОВИЙ ПРОЄКТ                                                                |                                       |   |           |         |
|                                     |               |          |        |      | Розробка програмного забезпечення для захисту від вірусів та шкідливих програм |                                       |   |           |         |
|                                     |               |          |        |      | Блок-схема алгоритму                                                           | Літера                                |   | Маса      | Масштаб |
| Зм.                                 | Арк.          | № докум. | Підпис | Дата |                                                                                |                                       | у |           |         |
| Виконав                             | Стафеев А. А. |          |        |      |                                                                                |                                       |   |           |         |
| Керівник                            | Олексів М. В. |          |        |      |                                                                                |                                       |   |           |         |
| Консульт.                           |               |          |        |      |                                                                                |                                       |   |           |         |
| Консульт.                           |               |          |        |      |                                                                                | Аркуш 1                               |   | Аркушів 3 |         |
| Зав. каф.                           | Рудик Ю. І.   |          |        |      |                                                                                | НУ «ЛП», ІКТА,<br>каф. ЕОМ, гр КІ-307 |   |           |         |
| Реценз.                             |               |          |        |      |                                                                                |                                       |   |           |         |

## Додаток В. Діаграма варіантів використання утиліти



|                                     |      |               |        |      |                                                                                |                                    |           |         |
|-------------------------------------|------|---------------|--------|------|--------------------------------------------------------------------------------|------------------------------------|-----------|---------|
| Міністерство освіти і науки України |      |               |        |      | КУРСОВИЙ ПРОЄКТ                                                                |                                    |           |         |
|                                     |      |               |        |      | Розробка програмного забезпечення для захисту від вірусів та шкідливих програм |                                    |           |         |
|                                     |      |               |        |      | Діаграма варіантів використання утиліти                                        | Літера                             | Маса      | Масштаб |
| Зм.                                 | Арк. | № докум.      | Підпис | Дата |                                                                                | у                                  |           |         |
| Виконав                             |      | Стафесв А. А. |        |      |                                                                                |                                    |           |         |
| Керівник                            |      | Олексів М. В. |        |      |                                                                                |                                    |           |         |
| Консульт.                           |      |               |        |      |                                                                                |                                    |           |         |
| Консульт.                           |      |               |        |      |                                                                                | Аркуш 2                            | Аркушів 3 |         |
| Зав. каф.                           |      | Рудик Ю. І.   |        |      |                                                                                | НУ «ЛП», ІКТА, каф. ЕОМ, гр КІ-307 |           |         |
| Реценз.                             |      |               |        |      |                                                                                |                                    |           |         |

Додаток Г. Діаграма класів програмного забезпечення



| Міністерство освіти і науки України |               |         |        |      | КУРСОВИЙ ПРОЄКТ                                                                |                                    |           |         |
|-------------------------------------|---------------|---------|--------|------|--------------------------------------------------------------------------------|------------------------------------|-----------|---------|
|                                     |               |         |        |      | Розробка програмного забезпечення для захисту від вірусів та шкідливих програм |                                    |           |         |
| Зм.                                 | Арк.          | №докум. | Підпис | Дата | Діаграма класів програмного забезпечення                                       | Літера                             | Маса      | Масштаб |
| Виконав                             | Стафесв А. А. |         |        |      |                                                                                | у                                  |           |         |
| Керівник                            | Олексів М. В. |         |        |      |                                                                                |                                    |           |         |
| Консульт.                           |               |         |        |      |                                                                                | Аркуш 3                            | Аркушів 3 |         |
| Консульт.                           |               |         |        |      |                                                                                |                                    |           |         |
| Зав. каф.                           | Рудик Ю. І.   |         |        |      |                                                                                | НУ «ЛП», ІКТА, каф. ЕОМ, гр КІ-307 |           |         |
| Реценз.                             |               |         |        |      |                                                                                |                                    |           |         |